

KNOWLEDGE TRANSFER DOCUMENT

RageV

Engine architecture, rendering engineering,
and the order it should be built in

A Windows game engine with two rendering backends, a scene editor, two scripting languages and a shipped game. This document explains what it is made of, how the pieces connect, what happens in a frame — and, most of all, the sequence in which a competent engineer should build these things so that each step is provable before the next one leans on it.

Subject RageV — C++20, Vulkan 1.3 and OpenGL 4.6, CMake

Audience Software engineers. No graphics background assumed; C++ fluency is.

Status at time of writing Phases 0–5 complete, phase 6 in progress. 888 automated checks.

Contents

0 How to read this

Who it is for · What it assumes · The one idea

1 What the engine is

Capability · Scale · The two applications

2 The mathematics, and what it is for

- 2.1 Vectors, and what the two products are actually for
- 2.2 Why 4×4 matrices, and what the fourth number is
- 2.3 The spaces a vertex passes through
- 2.4 The projection matrix, derived rather than copied
- 2.5 The depth buffer, and why z is not distance
- 2.6 Rotations: why the engine holds two representations
- 2.7 Normals do not transform like positions
- 2.8 Colour: linear light, sRGB storage, and HDR
- 2.9 The rendering equation, and what PBR actually computes
- 2.10 Alpha compositing, and why transparency is order-dependent
- 2.11 Frustum culling: six planes and one absolute value
- 2.12 Skinning: one equation and the test that validates it
- 2.13 The fixed timestep and the interpolation alpha

3 How the GPU and the two APIs actually work

- 3.1 What the hardware does
- 3.2 OpenGL: a global state machine
- 3.3 Vulkan: everything explicit, nothing implicit
- 3.4 The same idea in both APIs, side by side
- 3.5 A feature, followed all the way down through both

4 The build order — the spine of the document

- 4.1 Why order is the hard part
- 4.2 Phase 0 — the scene model, before anything is drawn
- 4.3 Phase 1 — assets and identity
- 4.4 Phase 2 — the loop, physics, scripts, audio
- 4.5 Phase 3 — the renderer, abstraction first
- 4.6 Phase 4 — projects, runtime, packaging
- 4.7 Phase 5 — scripting for real
- 4.8 The interlude that matters most — build a game
- 4.9 Phase 6 — what the game proved was missing

5 Architecture

- 5.1 Layers and the direction of dependency
- 5.2 The module boundary — engine as a DLL
- 5.3 The three "one description" systems

6 The frame

- 6.1 Fixed simulation, free rendering

6.2 The render path, pass by pass

6.3 The render graph

7 Rendering engineering

7.1 Designing an RHI over two very different APIs

7.2 Frames in flight, and everything that follows

7.3 Batching and instancing

7.4 Lighting: clustered forward

7.5 Shadows

7.6 Image-based lighting and reflection probes

7.7 Post-processing

7.8 Transparency, and why it is the hardest easy problem

7.9 Compute, and moving a simulation to the GPU

8 The systems around the renderer

8.1 Scene, entities, reflection

8.2 Assets

8.3 Scripting in two languages

8.4 Physics and audio

9 What experience taught — the load-bearing invariants

10 Verification: how any of this is known to work

11 Where it stands, and what comes next

12 Walkthroughs — building it yourself

12.1 Milestone 0 — a window, a device, and a triangle

12.2 Milestone 1 — the scene model, with nothing drawn

12.3 Milestone 2 — the loop everything else is written against

12.4 Milestone 3 — the RHI, proven by a second backend

12.5 Milestone 4 — the render graph and a multi-pass frame

12.6 Milestone 5 — it starts to look real

12.7 Milestone 6 — scripting, and hot reload

12.8 Milestone 7 — ship something

12.9 Milestone 8 — build a game, and let it tell you what is missing

12.10 What to build after that

A Appendix — repository layout, build, tools

0 How to read this

This is not an API reference and not a tour of a finished thing. It is an attempt to answer a harder question: if you had to build this, what would you build first, and what would you have to understand at each step to avoid painting yourself into a corner?

Who it is for

Somebody with real software engineering experience — comfortable in C++, comfortable with build systems, comfortable reading someone else's architecture — who has not built a renderer or an editor before. Graphics knowledge is not assumed; where a graphics concept is needed it is explained in the place it first matters, rather than in a chapter of preliminaries you would have forgotten by the time it was used.

What it assumes about the reader's goal

That you want a working engine rather than a rendering demo. Those are different projects and they diverge almost immediately. A rendering demo can hard-code its scene, load its assets from a fixed path, and put its camera in a global. An engine cannot do any of those things, and the difference is not a refactor you do later — it is the reason the build order below starts where it does.

The one idea, if you read nothing else

Build the thing that other things will be described in terms of, before you build the things. Identity before serialization. Serialization before the editor. Reflection before the inspector. The hardware abstraction before the renderer. The render graph before the passes. Every one of those pairs is cheap in that order and brutally expensive in the other, and the expense is not the code — it is that the second one has to be right in a hundred call sites you have already written.

The corollary is that the most valuable early work looks the least impressive. Weeks where the screen shows nothing new are the weeks that decide whether the project has a ceiling. This document tries to make those weeks legible.

A note on the code excerpts

Every snippet is taken verbatim from the engine, including its comments. The comments are load-bearing: in this codebase a comment usually records *why* a thing is not the obvious thing, and most of them exist because the obvious thing was tried first and failed in a way that took days to find. Where that is the case, the failure is described too.

1 What the engine is

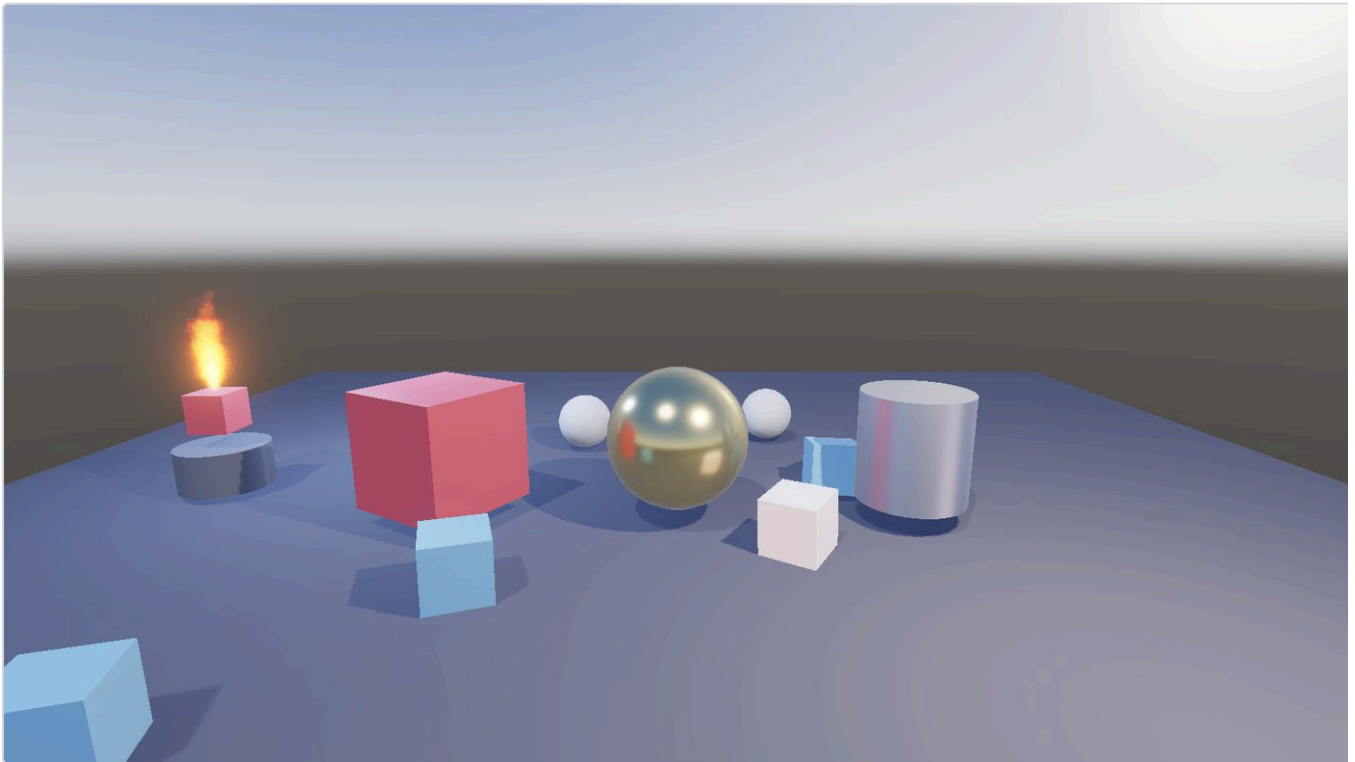


Figure 1. The sample project, rendered by the standalone runtime. Physically-based shading, a gold sphere reflecting a real-time reflection probe, cascaded shadow maps, image-based lighting from an HDR environment, and a GPU-textured particle flame on the pedestal. The boxes are Jolt rigid bodies that have fallen and settled.

Capability

RageV is a general-purpose 3D engine with an editor, targeting Windows. It renders through **either Vulkan 1.3 or OpenGL 4.6**, chosen at startup, from one set of engine code and one set of shaders. It has physically-based shading with image-based lighting, cascaded and omnidirectional shadows, real-time reflection probes, clustered forward lighting with no light count limit, frustum culling, instanced batching, skeletal animation, a CPU and GPU particle system, HDR post-processing, rigid-body physics, positional audio, and scripting in **both C++ and C#** with live reload in each.

It ships games: a project is a folder with its own assets, scenes, scripts and game module, and the editor can package one into a standalone folder that runs without the editor present.

Scale

Dimension	Figure	Note
Rendering backends	2	Vulkan 1.3 (dynamic rendering, no <code>VkRenderPass</code>) and OpenGL 4.6
Scripting languages	2	C++ game module DLL; C# via CoreCLR, both with hot reload
Automated checks	888	Run against both backends on every change
Frame, sample scene	0.80 ms Vulkan / 0.97 ms OpenGL	Release, 1600×900, vsync off
Frame, 1000 meshes	1.88 ms Vulkan / 1.59 ms OpenGL	Instanced batching; OpenGL wins here, and that is discussed
Frame, 256 local lights	3.15 ms Vulkan / 3.48 ms OpenGL	Clustered; 5.56 / 7.03 ms unclustered

The two applications

The engine is a DLL. Two executables link it: **RageVEDitor**, which has panels, gizmos, an inspector and play-in-editor; and **RageVRuntime**, which opens a project and plays it. They share the entire frame-building path and differ only in where the finished image is delivered — an imported viewport texture for the editor, the backbuffer for the runtime. Keeping that difference to one variable is what stops the two drifting into two renderers.

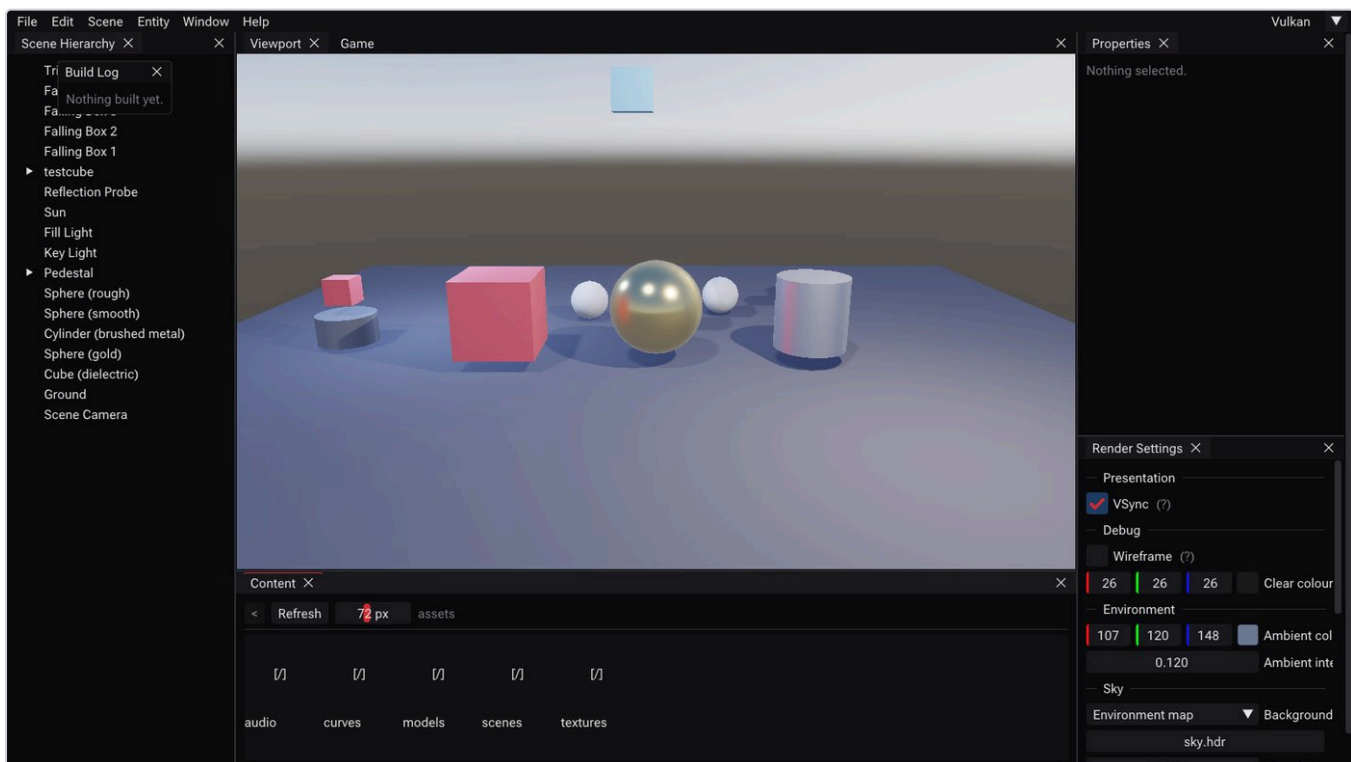


Figure 2. The editor. Scene hierarchy, viewport with a transform gizmo, reflection-driven inspector, content browser and render settings. Every field in the inspector is generated from the same component description that drives the scene file format and the C# bridge — see §3.3.

2 The mathematics, and what it is for

Everything in this chapter is standard, and none of it is hard. It is collected here because engine code assumes all of it silently, and reading a renderer without it is like reading a database without knowing what an index is — the code parses, and the reasons do not.

2.1 Vectors, and what the two products are actually for

You know what a vector is. What matters is that the two multiplication operations each answer one specific question, and essentially every use in a renderer is one of those two questions.

The dot product answers "how much do these two directions agree?" For unit vectors $a \cdot b = \cos \theta$: it is 1 when they point the same way, 0 when perpendicular, negative when opposed.

$$a \cdot b = a_x b_x + a_y b_y + a_z b_z = |a| |b| \cos \theta$$

That single fact is doing the work in: **diffuse lighting** (a surface facing the light receives light in proportion to $n \cdot l$ — the same energy spread over a larger area at a glancing angle); **backface culling** (is the triangle's normal pointing away from the camera?); **depth sorting** (a point's distance *along the view direction* is $(p - \text{eye}) \cdot \text{forward}$, which is what the particle renderer sorts by); and **frustum tests** (the signed distance from a point to a plane is a dot product).

The cross product answers "give me a direction perpendicular to both." Its main use in this engine is not geometry but *building coordinate frames*: given one axis, you need two more at right angles to it, and two cross products get you there. The camera's view matrix is built this way, and so is the cone the particle emitter spawns into:

```
// An orthonormal frame around the axis. The reference axis just has to not
// be parallel; Y fails only when the axis is Y.
const Vec3 reference = std::fabs(axis.y) < 0.99f ? Vec3(0.0f, 1.0f, 0.0f)
                                                : Vec3(1.0f, 0.0f, 0.0f);
const Vec3 tangent   = Math::Normalize(Math::Cross(reference, axis));
const Vec3 bitangent = Math::Cross(axis, tangent);
```

Note the guard. The cross product of two parallel vectors is the zero vector, which normalises to NaN and produces particles at infinity. Any time you build a frame from a single direction you need a fallback reference axis, and this is the idiom.

2.2 Why 4×4 matrices, and what the fourth number is

A 3×3 matrix can rotate, scale and shear, but it **cannot translate**: every such matrix maps the origin to the origin. Since moving things is rather central to a game engine, we need something more.

The trick — **homogeneous coordinates** — is to add a fourth component w and work in 4D. A 3D point (x, y, z) becomes $(x, y, z, 1)$ and a 3D *direction* becomes $(x, y, z, 0)$. Now the fourth column of a 4×4 matrix can hold a translation, and it is multiplied by w :

$$\begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} = \begin{bmatrix} x + wt_x \\ y + wt_y \\ z + wt_z \\ w \end{bmatrix}$$

translation matrix × the point = the result

A point ($w = 1$) is translated. A direction ($w = 0$) is not — which is exactly right.

This is not notation, it is a correctness rule you will use constantly. Transform a *position* with $w = 1$ and a *direction* with $w = 0$. Get it wrong and a normal or a velocity picks up the object's translation, which typically looks like geometry stretching toward the world origin. You can see the rule applied in the particle emitter, where the cone axis is a direction and the spawn point is a position:

```
// A direction: w = 0, so the emitter's position does not leak into it.
const Vec3 axis = local ? axisAuthorized
                  : Math::Normalize(Vec3(world * Vec4(axisAuthorized, 0.0f)));
// A position: the translation column of the matrix *is* the answer.
const Vec3 origin = local ? Vec3(0.0f) : Vec3(world[3]);
```

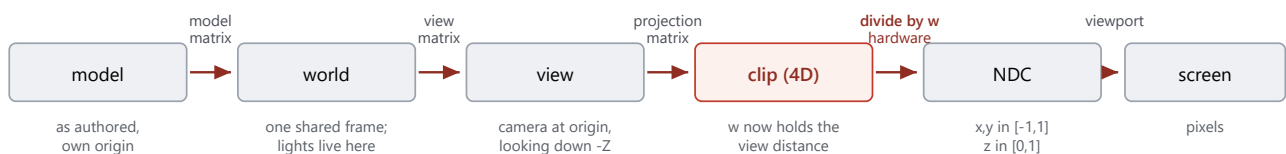
Composition is multiplication, and it does not commute. Rotating then translating is not the same as translating then rotating. The engine builds each entity's local matrix in the conventional scale → rotate → translate order:

```
Mat4 GetLocalTransform() const
{
    Mat4 rotation = Math::ToMat4(Math::FromEuler(Rotation));
    return Math::Translate(Mat4(1.0f), Position) *
           rotation *
           Math::Scale(Mat4(1.0f), Scale);
}
```

Read right to left, as the vector experiences it: scale the object about its own origin, then rotate it about its own origin, then move it. Any other order produces the thing every beginner sees once — an object that orbits the world origin when you scale it, or whose rotation is skewed by its scale.

2.3 The spaces a vertex passes through

A vertex makes the same journey every frame, through five coordinate systems. Almost every rendering bug is a value being used in the wrong one of these.



The only non-linear step

The projection matrix does not make distant things smaller. It arranges for w to hold the view distance, so that the divide does.

Diagram 1. The five coordinate systems a vertex passes through, and which matrix moves it between them. Almost every rendering bug is a value used in the wrong one of these.

Two observations that matter more than the diagram:

- **The view matrix is the inverse of the camera's world transform.** There is no "camera" in the maths — moving the camera left is identical to moving the entire world right, and that is literally what the matrix does.
- **The perspective divide is the only non-linear step**, and it is where perspective actually happens. The projection matrix does not make distant things smaller; it arranges for w to hold the view distance so that the divide does.

2.4 The projection matrix, derived rather than copied

Start from similar triangles. A point at view-space depth d (distance in front of the camera) and height y projects onto a screen plane at height y/d , scaled by how wide the lens is. With a vertical field of view fov , the half-height of the visible area at distance d is $d \cdot \tan(fov/2)$, so:

$$y_{\text{ndc}} = y / (d \cdot \tan(fov/2)) \quad x_{\text{ndc}} = x / (d \cdot \tan(fov/2) \cdot \text{aspect})$$

The division by d is the hardware's job, so the matrix's task is to put d into w and pre-scale x and y by the cotangent terms. In a right-handed view space the camera looks down $-Z$, so $d = -z$, which is where the -1 comes from:

```
Mat4 Perspective(float verticalFovRadians, float aspect,
                float nearPlane, float farPlane)
{
    // Right-handed, depth in [0, 1]. The [-1, 1] form in most textbooks
    // differs in [2].z and [3].z, and against a Vulkan-style clip volume it
    // throws away half the depth range while still producing a picture.
    const float tanHalfFov = std::tan(verticalFovRadians * 0.5f);

    Mat4 result(0.0f);
    result[0].x = 1.0f / (aspect * tanHalfFov);           // x scale
    result[1].y = 1.0f / tanHalfFov;                     // y scale
    result[2].z = farPlane / (nearPlane - farPlane);     // z remap
    result[2].w = -1.0f;                                  // w := -z == d
    result[3].z = -(farPlane * nearPlane) / (farPlane - nearPlane);
    return result;
}
```

`result[2].w = -1` is the entire trick: it copies $-z$ into w . The two z entries then remap depth into the range the hardware wants.

Depth ranges differ between APIs, and this is a real portability issue. OpenGL's traditional clip volume has z in $[-1, 1]$; Vulkan uses $[0, 1]$. Using the textbook $[-1, 1]$ matrix against a $[0, 1]$ volume throws away half your depth precision while still producing a plausible picture. This engine uses $[0, 1]$ for both and tells OpenGL to match with `glClipControl(GL_LOWER_LEFT, GL_ZERO_TO_ONE)`. Forget that call and every shadow comparison passes, and the scene renders uniformly lit.

2.5 The depth buffer, and why z is not distance

Work out what the matrix above does to depth. With $d = -z$:

$$z_{\text{ndc}} = f(d - n) / (d(f - n))$$

n and f are the near and far plane distances

That is a **hyperbola in d** , not a line. The consequence is worth seeing numerically, with this engine's usual near plane of 0.01 and far of 1000:

View distance d	z_{ndc}	Share of the buffer used so far
0.01 (near plane)	0.000000	—
0.1	0.900901	90% of the range in the first 0.1 units
1	0.990010	99%
2	0.995010	99.5%
20	0.999510	99.95%
200	0.999960	99.996%
1000 (far plane)	1.000000	100%

Two practical consequences follow, and both have bitten this engine.

Precision. Almost the entire buffer is spent on the first fraction of the range, so distant surfaces get very few distinguishable depth values and start to z-fight. The fix is not more bits, it is a *larger near plane*: the ratio f/n is what governs precision, so moving the near plane from 0.01 to 0.1 helps enormously and moving the far plane rarely helps at all.

Depth is not a distance and must never be used as one. This is precisely the bug described in §6.8: a published weighting formula used `gl_FragCoord.z` as though it varied usefully with distance. Look at the table — between 2 and 200 units it varies by 0.00005. The formula was numerically dead and the picture looked plausible anyway.

There is a clean way out, and it falls out of §2.4. Since w holds the view distance, and the hardware interpolates $1/w$ linearly across a triangle (which is what makes perspective-correct texturing work), the fragment shader is handed that reciprocal for free:

```
float d = 1.0 / gl_FragCoord.w;    →   linear view distance, in world units
```

2.6 Rotations: why the engine holds two representations

Euler angles are three numbers — yaw, pitch, roll — applied in some fixed order. They are what a human wants to type, and they are what the inspector shows. They have two failings: the order is a convention you must state, and at certain orientations two of the three axes align and you lose a degree of freedom (gimbal lock).

Quaternions are four numbers encoding an axis and an angle in a form that composes by multiplication, never gimbal-locks, and — crucially for animation — interpolates correctly. Interpolating Euler angles between two orientations swings through nonsense; spherical interpolation between quaternions takes the shortest arc at constant angular speed.

The engine's split follows from that: **components store Euler angles**, because they are authored by a person and serialized to a text file a person may edit; **composition and animation use quaternions**, converted at the boundary, as `GetLocalTransform` above shows. Store what humans edit, compute in what maths prefers, convert in one place.

2.7 Normals do not transform like positions

A surface normal is perpendicular to the surface, and perpendicularity does not survive non-uniform scaling. Squash a sphere along Y and the normals of a naively transformed mesh no longer point away from the surface — lighting goes subtly, and then obviously, wrong.

$$n' = (M^{-1})^T n$$

the inverse transpose of the model matrix, and for rotation-only matrices it equals M

The related engine-level version of the same idea: a particle emitter in local space scales its particles by the *longest basis axis* of its transform rather than by each axis separately, because a particle is a round thing and a non-uniform scale has no single correct answer for it.

2.8 Colour: linear light, sRGB storage, and HDR

This is the topic that produces the most "why does it look washed out" hours, and the rule is short: **light adds linearly, but images are not stored linearly**.

Displays and image files use roughly a $\gamma \approx 2.2$ curve (sRGB), which spends more of its 8 bits on darks, where the eye is more sensitive. Physical light does not work that way: two lamps are exactly twice one lamp. So the pipeline must be:

1. **Decode on read.** Colour textures are sRGB and are converted to linear when sampled — done by the hardware if the texture format says `_SRGB`. This is why the engine loads sprites and albedo maps as sRGB and *data* maps (roughness, normals, masks) as linear: a normal map is not a picture and decoding it would corrupt it.
2. **Do all lighting in linear space**, in a floating-point buffer. Values are not confined to $[0, 1]$ — the sun is not "1.0 bright" — and this is what HDR means here.
3. **Tonemap once, at the end**, mapping the unbounded linear range into displayable $[0, 1]$ with a filmic curve (ACES), then encode back to γ .

Hence a load-bearing invariant of the shading code: **the PBR shader outputs linear HDR only**. Any pass that applies a tone curve early makes every later pass wrong, and the error compounds silently because each individual image still looks like an image.

2.9 The rendering equation, and what PBR actually computes

All of shading approximates one integral: the light leaving a point toward the eye is the sum, over every incoming direction, of light arriving times the fraction of it the surface sends toward the eye.

$$L_o(v) = \int_{\Omega} f_r(l, v) L_i(l) (n \cdot l) d\omega$$

f_r is the BRDF — the surface's answer to "given light from l, how much goes to v?"

Note the $n \cdot l$ from §2.1 sitting inside it. "Physically based" means choosing an f_r that conserves energy and is derived from a model of the surface rather than from artistic knobs. This engine uses Cook-Torrance, whose specular term is:

$$f_{\text{spec}} = D(h) \cdot F(v, h) \cdot G(l, v) / (4 (n \cdot l)(n \cdot v))$$

The three terms model a surface as a field of microscopic mirrors:

- **D — distribution** (GGX). What fraction of those micro-mirrors are angled to bounce light from l into v ? This is what roughness controls, and it is the shape of the highlight.
- **F — Fresnel** (Schlick's approximation). Everything becomes more reflective at grazing angles — the reason a road looks like water toward the horizon.
- **G — geometry** (Smith). Micro-mirrors shadow and mask each other at glancing angles; this term removes energy that would otherwise be double-counted.

Metallic/roughness is the artist-facing parameterisation of that. Metals have no diffuse component and tint their reflection with their own colour; dielectrics have a diffuse colour and a colourless ~4% reflection. Rather than expose those as separate knobs, one **Metallic** value blends between the two behaviours, and **Roughness** drives D .

Why image-based lighting needs three precomputed things

The integral above has to be evaluated against the *whole environment*, not just a few point lights, and you cannot do that per pixel per frame. The standard split-sum approximation factors it into parts that can be precomputed, and this is exactly why the engine builds three artefacts from one environment map (§7.6):

- an **irradiance cube** — the diffuse half, integrated over the hemisphere once;
- a **prefiltered cube with roughness in its mips** — the specular half, where rougher means a blurrier mip, so a roughness value becomes a mip level;
- a **2D BRDF lookup table** — the remaining scale and bias, which depend only on roughness and $n \cdot v$ and so are the same for every scene.

2.10 Alpha compositing, and why transparency is order-dependent

Blending a translucent fragment over what is already there is the **over** operator:

$$C_{\text{result}} = \alpha_s C_s + (1 - \alpha_s) C_d$$

Apply it twice, in the two possible orders, and the results differ — the operator is not commutative. **That single fact is the whole difficulty of transparency.** Opaque geometry escapes it because the depth test discards all but the nearest fragment, so order stops mattering; translucent geometry has no such escape and must either be sorted or use a formulation that does not care.

Additive blending is the trivial escape: $C = C_s + C_d$ is commutative because addition is. That is why fire and sparks are cheap and always correct, and why anything that reads as *matter* rather than *light* is not.

Weighted-blended OIT (§6.8, §7.8) takes the other route — two accumulators that are each order-independent by construction:

$$\text{accum} = \sum w_j \cdot (C_j \alpha_j \alpha_j) \quad \text{reveal} = \prod (1 - \alpha_j)$$

$$C = (\text{accum.rgb} / \text{accum.a}) \cdot (1 - \text{reveal}) + C_{\text{background}} \cdot \text{reveal}$$

A sum and a product: neither cares about order. Now look at what happens if every w_i is the same constant k . It factors out of both the numerator and the denominator of $\text{accum.rgb} / \text{accum.a}$ and **cancels exactly** — the weighting silently becomes a plain average. That is not a hypothetical; it is the bug §7.8 describes, and this equation is why replacing the weight with the literal `1.0` produced a pixel-identical image.

2.11 Frustum culling: six planes and one absolute value

The visible region is a truncated pyramid bounded by six planes. Those planes can be extracted directly from the combined view-projection matrix by adding and subtracting its rows — a point is inside the frustum exactly when its clip coordinates satisfy $-w \leq x, y \leq w$ and $0 \leq z \leq w$, and rearranging those inequalities gives the plane equations.

Testing a box against a plane looks like it needs eight corner tests. It does not. For a box with centre c and half-extents e , the projection of the box onto the plane normal has radius:

$$r = e_x |n_x| + e_y |n_y| + e_z |n_z|$$

and the box is outside when the centre's signed distance is less than $-r$

The absolute values are the entire trick, and the same trick reappears when a rotated box needs a world-space bounding box: taking the absolute value of the transform's basis rows gives a box that contains the rotated one. The engine's `TransformBounds` does exactly this, which is why a rotated particle quad still gets a correct bound.

2.12 Skinning: one equation and the test that validates it

A skinned vertex is attached to several bones with weights summing to 1. Each bone contributes the vertex transformed *out of* bind pose and *into* the bone's current pose:

$$v' = \sum_i w_i \cdot M_i \cdot B_i^{-1} \cdot v$$

B_i^{-1} is the inverse bind matrix; M_i is the bone's current world matrix

The inverse bind matrix is the piece newcomers omit or misplace, and its meaning is simple: it moves the vertex from model space into the bone's local space *as the mesh was authored*, so that the bone's current transform then means "relative to where I started".

That gives a beautiful test, and it is the one the engine uses. At the bind pose, $M_i = B_i$, so every product must be the identity. One assertion — every bone's skinning matrix is the identity at bind pose — simultaneously catches wrong inverse binds, wrong multiplication order, and matrices computed in the wrong space.

2.13 The fixed timestep and the interpolation alpha

Physics must advance in equal steps or it is not reproducible: a heavier frame would mean a larger step, a larger integration error, and a different outcome. But frames do not arrive at equal intervals. The standard resolution is an accumulator:

```
accumulator += min(frameTime, 0.25)    // clamped, so a stall is forgiven
while accumulator >= dt:                // zero or more times
    simulate(dt)
    accumulator -= dt
alpha = accumulator / dt                // in [0, 1): how far into the next step
```

The clamp matters: without it, a two-second stall would try to simulate two seconds of physics in one frame, taking even longer, and the loop never recovers — the "spiral of death". Forgiving the debt is the standard, correct choice.

alpha is the interesting part. At 240 Hz with a 60 Hz simulation, three frames in four run *zero* steps. Rendering the simulation state directly would show the same positions for four frames and then jump. Instead the renderer blends the previous and current simulation states:

$$p_{\text{render}} = p_{\text{previous}} \cdot (1 - \alpha) + p_{\text{current}} \cdot \alpha$$

This is why `Application::GetInterpolationAlpha()` exists and why physics transform sync happens per *frame* rather than per step — and why a script that moves a camera on the fixed step fights the interpolation rather than benefiting from it (§6.1).

3 How the GPU and the two APIs actually work

You cannot design an abstraction over Vulkan and OpenGL without knowing what each is abstracting. This chapter is the mental model, and it ends with one feature followed all the way down through both.

3.1 What the hardware does

A GPU is a machine for running the same small program over a very large number of independent items. A draw call feeds it in this shape:

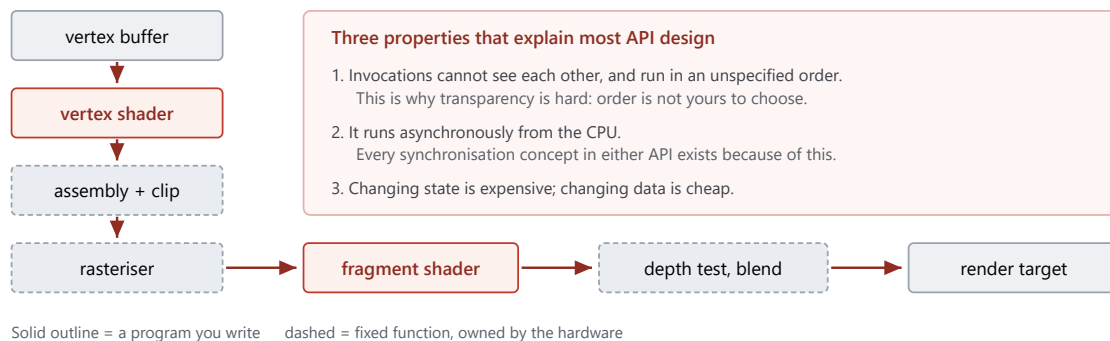


Diagram 2. The graphics pipeline. You write two of these stages; the hardware owns the rest.

Three properties of this machine explain most API design decisions:

- **Invocations cannot see each other and run in an unspecified order.** That is what makes it fast and why transparency (§2.10) is hard: order is not yours to choose.
- **It runs asynchronously from the CPU.** You submit work and it completes later. Every synchronisation concept in either API exists because of this one fact.
- **Switching state is expensive; changing data is cheap.** That is why draws are batched by pipeline state (§7.3) rather than by object.

3.2 OpenGL: a global state machine

OpenGL models the GPU as one enormous global object with hundreds of settings. You mutate it with calls, then say "draw", and the draw uses whatever state happens to be set. There is no handle for "the pipeline" — the pipeline is the current state.

```
// OpenGL: bind, mutate, draw. The object being edited is whatever was
// bound most recently, which is why order-of-calls is load-bearing.
glBindFramebuffer(GL_FRAMEBUFFER, fbo);
glDrawBuffers(2, buffers);           // which attachments outputs go to
glEnable(GL_BLEND);
glBlendFunci(0, GL_ONE, GL_ONE);     // per-attachment blend
glUseProgram(program);
glBindVertexArray(vao);
glDrawElements(GL_TRIANGLES, count, GL_UNSIGNED_INT, nullptr);
```

The driver handles memory allocation, residency and synchronisation for you. That is genuinely convenient and it is also the problem: those decisions are made by heuristics you cannot see, on a thread you do not control, and when they are wrong you have no lever. The state machine also has a specific failure mode worth naming, because it caused a real bug here:

Global state persists into whatever runs next. `glDrawBuffers` is framebuffer state, not draw state, so a pass that selected a subset of attachments leaves the *next* pass writing into that selection. The engine now sets it unconditionally on every pass. A related trap: with a subset selected, `glClearBufferfv` indexes into the *draw-buffer list*, not the framebuffer — selecting attachments 1 and 2 means clearing indices 0 and 1.

3.3 Vulkan: everything explicit, nothing implicit

Vulkan removes the driver's judgement and hands you the decisions. There is far more code, and in exchange nothing is guessed on your behalf. The concepts you must know:

Concept	What it is, and why it exists
Physical / logical device	The GPU, and your connection to it. You query features (§3.5 shows why) and enable only what you use.
Queue	Where work is submitted. Work is a <i>list</i> you build, not a series of calls that execute.
Command buffer	A recorded list of GPU commands. Recording is CPU work; submission is when the GPU sees it. This is the single biggest departure from OpenGL.
Memory + buffers/images	You allocate memory and bind resources to it, choosing <i>device-local</i> (fast for the GPU) or <i>host-visible</i> (the CPU can write it). Getting this wrong is a performance cliff, not an error — see §7.9.
Descriptor set	A pre-baked bundle saying "this buffer is binding 0, that texture is binding 1". Replaces OpenGL's bind-before-draw with bind-a-set-of-bindings.
Pipeline	An immutable object baking shaders, blend state, depth state and vertex layout together. Created up front so the driver compiles once instead of guessing at draw time.
Render pass / dynamic rendering	Declares the attachments being drawn into. Vulkan 1.3's <i>dynamic rendering</i> lets you declare them inline instead of building objects ahead of time — this engine uses it exclusively, which is what makes §6.3's per-pass attachment subsets natural.
Fence / semaphore / barrier	CPU waits for GPU; GPU waits for GPU; and "this memory was written by X and will be read by Y". Nothing is implicit.
Swapchain	The images you present to. Owned by the presentation engine, which is a separate thing from the queues — a distinction that causes a specific crash, below.

The mental shift is from *calls that do things* to *lists that are submitted*. Most Vulkan bugs are lifetime bugs: something was destroyed or rewritten while a submitted list still referred to it, and the symptom appears frames later. This is the origin of the frames-in-flight discipline in §7.2, and of an invariant that reads oddly until you know the model:

A swapchain is retired, not destroyed and replaced. Pass the old handle as `oldSwapchain` and destroy it only after the new one exists — `vkDeviceWaitIdle` waits for the *queues*, and the presentation engine is not a queue. This crashed on unchecking VSync.

3.4 The same idea in both APIs, side by side

Idea	OpenGL	Vulkan
Shader program	<code>glCreateProgram</code> , GLSL compiled by the driver	SPIR-V module, compiled offline or at load
Draw state	Whatever is currently set	An immutable pipeline object
Binding resources	<code>glBindTextureUnit</code> , <code>glBindBufferBase</code>	Descriptor sets, bound as a group
Render target	FBO + <code>glDrawBuffers</code>	Dynamic rendering attachment list
Uploading	<code>glBufferSubData</code> — driver decides how	Staging buffer + explicit copy + barrier
Synchronisation	Implicit; <code>glMemoryBarrier</code> for image/SSBO writes	Explicit barriers, semaphores and fences
Image origin	Bottom-left	Top-left — see §7.7, this one is a menace

3.5 A feature, followed all the way down through both

Take **weighted-blended transparency** (the maths is §2.10). It is a good worked example because it touches every layer: shaders, pipeline state, render targets, per-attachment blending, a device feature, and a resolve pass. Here is the whole descent, from what the engine asks for to what each API does.

1 — What the feature needs

Two extra colour attachments alongside the scene's HDR colour, sharing its depth buffer so particles depth-test against solid geometry; particles drawn once, writing two outputs with *different blend equations per attachment* (accumulation adds, revealage multiplies); and a fullscreen pass that reads both and composites.

2 — The shader writes two outputs

```
// Attachment 1: the running sum of premultiplied colour, weighted by depth.
layout(location = 0) out vec4 o_Accumulate;
// Attachment 2: how much background is still showing.
layout(location = 1) out float o_Revealage;

// NOT gl_FragCoord.z -- see 2.5. w holds the view distance.
float d = 1.0 / max(gl_FragCoord.w, 1e-6);
float weight = color.a * clamp(kUnitWeightDistance / max(d, 1e-3), 1e-2, 3e3);

o_Accumulate = vec4(color.rgb * color.a, color.a) * weight;
o_Revealage = color.a; // blend equation dst *= (1 - src) makes this a product
```

Note that `location` here means "the *n*th attachment this pass bound", not "attachment *n* of the target". That indirection is the source of two of the traps below.

3 — The RHI request

The engine asks in API-neutral terms: a graphics pipeline whose colour formats are the two attachment formats, with per-attachment blend presets.

```
GraphicsPipelineDesc desc;
desc.ColorFormats      = { Format::R16G16B16A16_SFLOAT, Format::R8_UNORM };
desc.BlendPerAttachment = { BlendPreset::WeightedAccumulate, // ONE, ONE
                           BlendPreset::WeightedRevealage }; // ZERO, ONE_MINUS_SRC
desc.DepthStencil.DepthTestEnable = true; // against the scene's depth
desc.DepthStencil.DepthWriteEnable = false; // particles do not occlude each other
```

4a — What Vulkan does with that

The blend state is baked into the immutable pipeline object at creation, one `VkPipelineColorBlendAttachmentState` per attachment. And here the API imposes something OpenGL does not:

```
std::vector<VkPipelineColorBlendAttachmentState> blendAttachments(
    std::max<size_t>(1, m_Desc.ColorFormats.size()));

// Attachments may only differ from one another where the device allows it.
// Without independentBlend, Vulkan requires every attachment to match the
// first, so honouring the request would be a validation error rather than
// a wrong picture.
const bool perAttachment = !m_Desc.BlendPerAttachment.empty()
    && m_Device.IndependentBlendSupported();
```

independentBlend is an optional device feature. It must be queried and enabled at device creation, and if it is missing the engine falls back to attachment zero's preset and warns, naming the pipeline — a wrong picture rather than a crash. This is the Vulkan pattern in miniature: capabilities are asked for, not assumed, and the validation layer tells you immediately when you assumed.

At draw time the pass declares its attachments inline (dynamic rendering), which is what makes binding attachments 1 and 2 of a three-attachment target a natural thing to express rather than a new framebuffer object.

4b — What OpenGL does with that

There is no pipeline object, so the same information is applied as state at bind time: `glEnablei / glBlendFunci` per draw buffer. No feature query is needed — indexed blending is core in 4.x. But the framebuffer must be told which attachments the outputs map to, and that is persistent state:

```
// glDrawBuffers decides which attachments the fragment shader's outputs
// land in, and the clears below address them by index into *this* list
// rather than into the framebuffer -- selecting attachments 1 and 2 means
// the shader's location 0 writes attachment 1.
for (const ColorBinding& binding : info.ColorAttachments)
    buffers.push_back(GL_COLOR_ATTACHMENT0 + binding.Index);

if (!buffers.empty())
    glDrawBuffers((GLsizei)buffers.size(), buffers.data());
```

5 — The resolve, and the bug that hid in it

A fullscreen triangle samples both accumulators and composites (§2.10). It is one pass, and one is an odd number — which is exactly the condition under which the image-origin difference in §3.4 stops cancelling out. Vulkan composited every weighted particle upside down for an entire release, because flipped smoke still looks like smoke. §7.7 has the full account; the point here is that **the disagreement between the two APIs was not in the feature, it was in a convention two layers below it.**

What this example is meant to teach

- **The same feature is "bake it into an object" on one API and "set it before the draw" on the other.** An RHI that models the *request* rather than either mechanism can serve both; one modelled on either API's mechanism cannot.

- **Vulkan tells you when you assumed a capability; OpenGL usually does not.** Developing against Vulkan with validation on and porting to OpenGL catches more than the reverse.
- **The differences that hurt are conventions, not features.** Image origin, depth range and instance-index base each produce a correct picture on one backend and a wrong one on the other, with no error anywhere. This is the entire argument for comparing both backends' output as a standing practice (§10).

4 The build order

This is the part worth arguing with. Everything else in the document describes a state; this describes a sequence, and the sequence is the transferable knowledge.

Chapter 12 walks this chapter. Here is the argument for the order; there are eight milestones that execute it, each ending with something running and a check that proves it.

4.1 Why order is the hard part

Most engine tutorials are organised by subsystem: here is a renderer, here is a physics integration, here is an editor. That organisation is fine for explaining a subsystem and actively misleading as a plan, because it hides the dependencies that actually bite. Those are rarely compile-time dependencies. They are *format* and *identity* dependencies, and they bite when you try to change something after a hundred files already contain it.

Two rules produced the order below.

Rule 1 — describe before you draw

An engine is a program for describing scenes. Rendering is one consumer of that description; serialization, the editor, the undo stack, prefabs, the network layer you have not written, and every script are the others. Build the description first, make it persist losslessly, and only then draw it. A renderer built first will invent its own notion of a scene, and every later system will be a translation layer.

Rule 2 — abstraction before the features that would hard-code themselves

There are exactly two places where retrofitting an abstraction costs more than writing it up front: the graphics API boundary and the frame's pass structure. Both are touched by every rendering feature you will ever add. Write the RHI before the second backend is a thought, and write the render graph before the third pass. Anywhere else, abstract late.

4.2 Phase 0 — the scene model, before anything is drawn

Deliverable: a scene you can build in an editor, save, load, and get back byte-identical, with undo.

Concretely, in this order:

1. **An editor camera.** Trivially small, and it means every later step is visible.
2. **Entity UUIDs.** Not the ECS library's handles — those are recycled and mean nothing outside one registry instance. A stable 64-bit identity on every entity, assigned at creation, preserved through save, load and play mode. *This is the single highest-leverage decision in the project* and it costs an afternoon. Retrofitting identity later means touching prefabs, scripts, the undo stack, the physics body map and every serialized reference at once.
3. **Transform hierarchy,** with parent stored by UUID and children rebuilt on load. Derived state that can disagree with its source is the most common bug class in a scene graph; storing only the parent link

makes disagreement impossible.

4. **Component reflection** — a registry that describes each component's fields. Then write the serializer *against the registry*, and the inspector against it afterwards. @@@@§9.3 covers why this single decision pays for itself repeatedly.
5. **An undo/redo command stack.** Early, because commands are how the editor mutates the scene, and retrofitting that means rewriting every editor action.
6. **A lossless round-trip test.** Build a scene, serialize, deserialize, serialize again, compare the strings. This one test will catch more real bugs over the project's life than any other, and it is twenty lines.

Why not draw first? Because the moment you can draw, you will want to draw more, and everything you add will be described in whatever ad-hoc way the renderer already understands. The scene model is the vocabulary. Fix the vocabulary first.

4.3 Phase 1 — assets and identity

Deliverable: a scene file that references a mesh without containing it.

Assets get the same treatment entities did: a 64-bit `AssetHandle`, a registry that maps handles to paths, and a `.meta` sidecar beside each source file that records the handle so it survives a rename. Then a real importer — glTF, via `cglTF` — and meshes, materials and textures as addressable assets rather than things a loader hands you.

The ordering point: **import comes after handles**. If you write the importer first it will return objects, and you will spend the next phase converting object returns into handle returns everywhere.

4.4 Phase 2 — the loop, physics, scripts, audio

Deliverable: a scene that *runs*, and reverts perfectly when you stop.

1. **The fixed-timestep loop first.** Before physics, before scripts. It is the contract both will be written against, and @@@@§10.1 explains why it cannot be retrofitted cheaply.
2. **Play/edit split with snapshot restore.** Play mode operates on a copy; stopping restores the original exactly. Doing this before scripts exist means scripts never get to assume they can mutate the authored scene.
3. **Input as named actions**, never key codes. A script written against `"Jump"` survives rebinding; one written against a key code does not, and by the time you notice, fifty scripts have key codes in them.
4. **Scripts, then physics, then contact callbacks.** In that order within the step, and documented as such — see @@@§10.4.
5. **Audio** last of the four, because it is the only one nothing else depends on.

4.5 Phase 3 — the renderer, abstraction first

Deliverable: the picture in Figure 1, on two backends, from one set of code.

This is the largest phase and the one most likely to be attempted in the wrong order. The correct order is counter-intuitive because it front-loads two pieces of pure plumbing that render nothing new.

1. **The RHI.** A device, command list, buffer, texture, pipeline, shader, resource set and sampler abstraction, with two implementations. Writing this against a single backend is a trap: you will abstract the wrong things, because the shape of the API you have is invisible to you until a second one disagrees with it. @@§9.1 covers what to abstract and — as importantly — what not to.
2. **The render graph.** Passes declare the targets they read and write; the graph allocates and pools them and orders the passes. Written when the frame had three passes, which felt premature; the frame now has eleven, and adding a pass is a declaration rather than a surgery. @@@@§10.3.
3. **Then features,** in roughly this order, because each depends on the previous: cube maps and sky → environment reflections → image-based lighting → shadows → culling → instanced batching → clustered lighting → skeletal animation.

The measurement discipline starts here. Two of the optimisations in this phase — frustum culling and instanced batching — cut draw calls dramatically and produced *no* frame-time improvement whatsoever. Culling took 144 draws to 60; instancing took 3238 to 854; the clock did not move either time. A draw count is not a frame time. See §10.

4.6 Phase 4 — projects, runtime, packaging

Deliverable: a folder you can send someone that runs without the editor.

Doing this before the engine is "finished" is deliberate. Packaging is where you discover that your asset paths were relative to the editor's working directory, that your shaders were loaded from the source tree, and that the runtime renders a blank window because a pass clears the backbuffer after the scene is drawn — a bug with a perfectly healthy draw-call count, which is why the RHI has a screenshot capture (§10).

4.7 Phase 5 — scripting for real

Deliverable: the same game logic, in C++ or C#, either reloadable without restarting.

C# is hosted through CoreCLR, with the managed assembly loaded from *bytes* into a collectible `AssemblyLoadContext` that is retired and replaced on every build. The unload is verified with a weak reference; a context that will not die is reported rather than ignored, because a leaked context means the old code is still running and the symptom is "my change did nothing".

4.8 The interlude that matters most — build a game

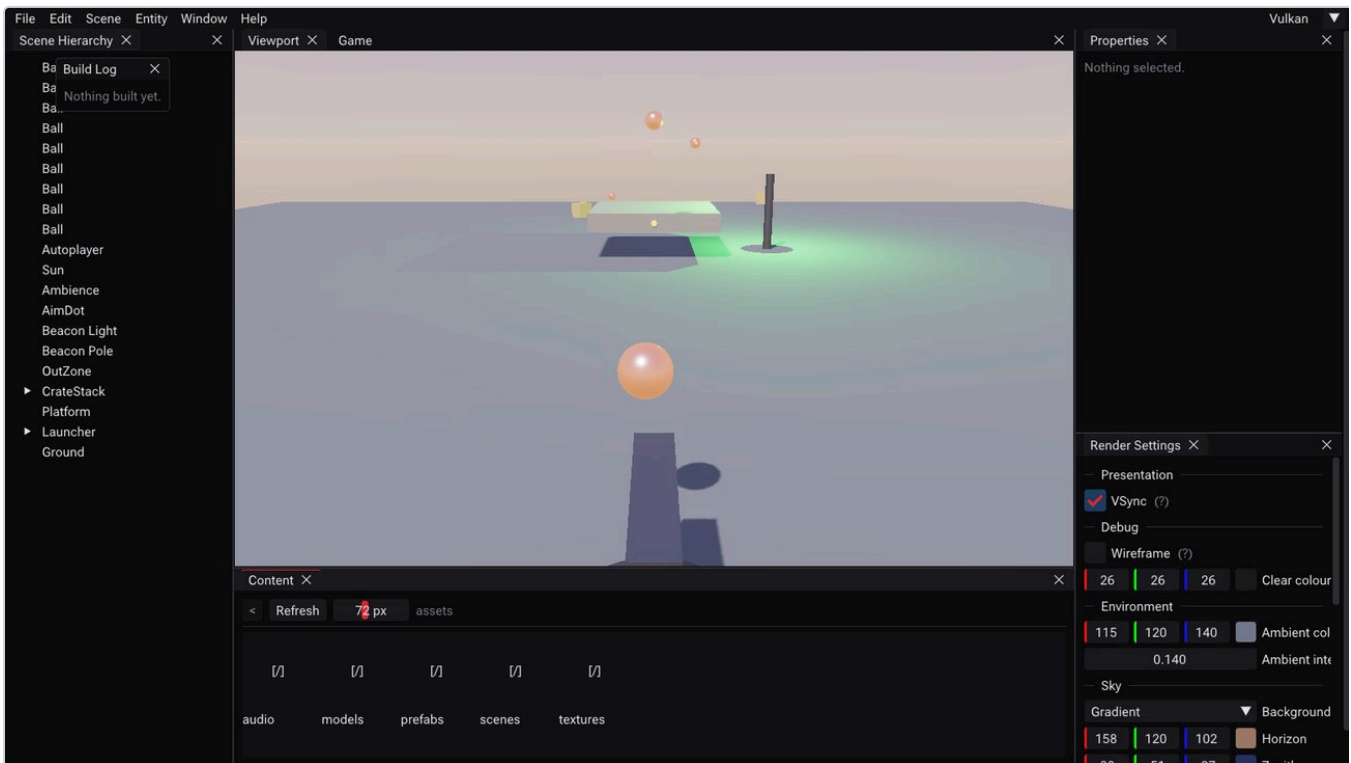


Figure 3. *Knockdown* — the mini game built to find out what the engine was missing. Aim a launcher, lob balls at a crate pyramid, knock them into the scoring zone. Deliberately split across both languages: C++ does input, aiming and spawning; C# does scoring, the component bridge and audio.

Building a small, complete, shippable game before adding more engine features is the highest-value scheduling decision in the whole project. It found **six real engine holes in one session**, every one of which was invisible from inside the engine:

- A menu item that had been implemented but never wired into the menu.
- A rigid body added during play never joined the simulation — `AddBody` existed and had no callers. The first ball fired hung in the air.
- C# collision callbacks were never delivered from a real simulation. The entire managed contact surface was written, bound, documented and tested at the table, and unreachable.
- Nothing outside the test suite ever booted the .NET host, so in a fresh editor C# was dead until a manual build, and a packaged game had no C# at all.
- Editor C# builds always failed, because a reference path was resolved against the wrong directory.
- A deferred-deletion race that freed descriptor sets while a frame was still in flight — found later, but by the churn the game created.

Note the pattern: *none of these were bugs in a feature*. They were bugs in the seams between features, and only an application that crosses the seams finds them. A test suite written by the same person who wrote the feature tends to test the feature.

4.9 Phase 6 — what the game proved was missing

The game produced a ranked list of what actually hurt, which is a far better backlog than a list of what sounds impressive. In order: **no text or UI** (the game communicated entirely through a light and four sounds), **no**

per-frame script hook (camera smoothing and recoil had nowhere to live), **no particles**, and some small API gaps in audio.

Particles were built first at the user's direction, and are covered in @@§9.8 and @@§9.9. Text and UI, and the per-frame hook, remain open.

5 Architecture

5.1 Layers and the direction of dependency

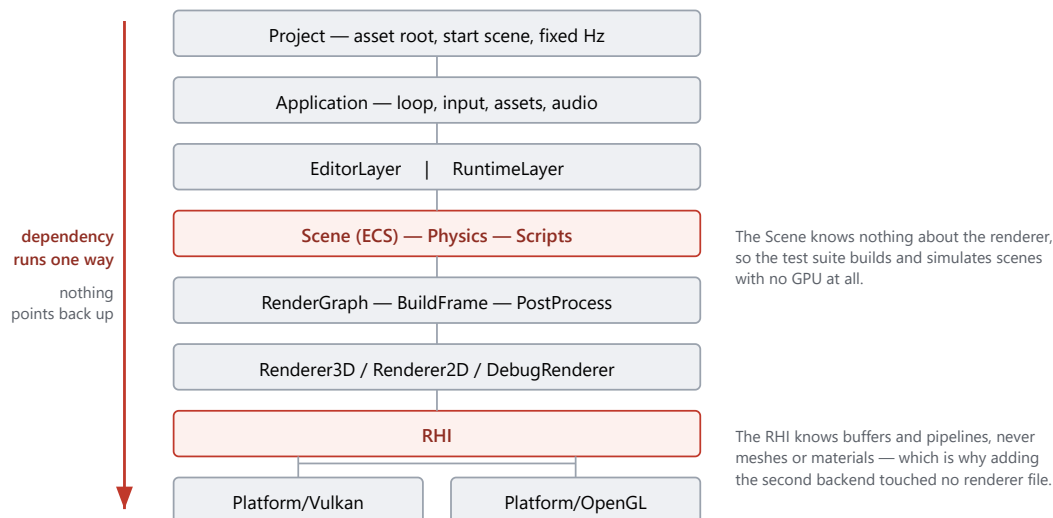


Diagram 3. The layers, and the constraint that gives the picture its meaning: dependency runs strictly downward. The two highlighted boundaries are the ones that pay for themselves.

The rule that makes the diagram mean something

Boxes and arrows are cheap. The content of this one is a constraint: **dependency runs strictly downward, and nothing points back up.** The `Scene` knows nothing about the renderer. The renderer reads the scene and never writes it. The RHI knows nothing about scenes, meshes or materials — it knows buffers, textures and pipelines.

That is easy to state and constantly tempting to violate. The temptations are specific, and each has an alternative:

The temptation	Why it appeals	What to do instead
A component holding a GPU buffer	The renderer needs per-object GPU data anyway	Key the buffer by the entity's UUID in a renderer-side cache. The scene stays loadable without a device.
The renderer writing a "visible" flag back onto entities	Culling has to record its answer somewhere	Keep the answer in the renderer's own per-frame list. A scene that changes when you look at it cannot be reasoned about.
The RHI knowing what a material is	It would make the draw call shorter	Let the renderer translate material → pipeline + resource set. The RHI's vocabulary is the thing keeping two backends honest.

What the discipline actually buys

Two things, both of which paid for themselves many times over.

The test suite runs without a GPU. Because nothing in the scene layer needs a device, the suite can build scenes, parent entities, run the simulation, serialize, reload and assert — hundreds of checks — with no window and no driver. If a `TransformComponent` held a GPU buffer, every one of those tests would need a device, and the ones that run in CI would not exist.

The second backend was additive. Adding OpenGL touched no renderer file. That is the observable consequence of the RHI boundary being in the right place, and it is the test you should apply to your own: *if I added a second implementation of this layer, how many files above it would change?* The answer should be zero, and if it is not, the abstraction is leaking in a direction you have not noticed yet.

Layers, in the other sense

The word is overloaded and worth disambiguating, because the engine uses it for a second thing. A `Layer` is also a runtime object with `OnUpdate`, `OnFixedUpdate`, `OnImGuiRender` and event handlers, held in a stack by the `Application`. `EditorLayer` and `RuntimeLayer` are the two that matter. It is the mechanism by which the same engine core drives two different applications, and it is why the frame loop in §6.1 can be written once.

Layer's destructor must stay virtual. The stack owns layers as `Layer*` and deletes through that pointer. Without a virtual destructor only `~Layer` runs, and every derived member leaks — in this case the editor's scene, its render targets, and every material in them. Undefined behaviour that survived months, because nothing looked wrong: the leak only appeared at shutdown, and shutdown is where nobody is looking.

5.2 The module boundary — engine as a DLL

Why a DLL at all

The engine builds as `RageV.dll`, and the editor, runtime, test tools and each project's game module link against it. That is not an aesthetic choice. It is what makes **per-project C++ scripting** possible: a project's `Source/` folder compiles into its own module DLL that links the engine, and the editor loads and unloads it. Without the boundary, changing a script would mean rebuilding the engine, and hot reload would be impossible.

The price is that a module boundary is a real thing with real rules, and two of them are invisible until violated.

Rule one: statics do not cross

Static data read through an inline accessor does not cross a module boundary. The accessor is inlined into the consumer, which then links its own copy of the data. You get two "singletons" — the engine's and the module's — and every write through one is invisible to the other. Anything genuinely global needs an *out-of-line*, exported accessor. In this engine `Application::Get`, the two loggers and the input state all had to be converted.

The symptom is memorable: the module's log messages vanish, or its `Application::Get()` returns something uninitialised, while the identical code compiled into the engine works perfectly.

Rule two: registrars can be dropped entirely

A translation unit whose only contents are static registrars will be dropped from a static library. The linker sees no referenced symbol in that object file and omits it, taking your registration with it. It no longer

bites the engine, which is a DLL and therefore linked from its object files — but it is why built-in scripts are still registered by an explicit function call, and it would bite again the moment anything moved back into a `.lib`.

Rule three, learned late: do not link the same vendored library twice

This one generalises past graphics and past C++, and it produced two symptoms that looked completely unrelated.

`RageV.dll` links GLFW *privately*. The RHI smoke-test tool linked GLFW again, into its own executable, and called `glfwInit` and `glfwCreateWindow` itself. GLFW keeps its state in globals, so the executable and the DLL each had their own — **the window was real, and the engine's copy had never heard of it.**

- On **Vulkan**: `glfwGetWin32Window` inside the DLL returned null, so surface creation failed with "hwnd is NULL".
- On **OpenGL**: it failed one step earlier and looked like a different bug entirely — glad was handed a proc-address loader belonging to a library nobody had initialised, and reported that it could not load GL functions.

One cause, two symptoms with no visible relationship. When two failures in different subsystems appear at the same time, the shared cause is usually one layer below both. And the fix already existed and had not been applied: a `Visible` flag on the engine's own window class, added for exactly this situation, whose comment described this exact failure. The tool had never been migrated onto it.

A fix that leaves the old path compiling leaves it in use. If you solve a problem by adding the right way, you have not finished until the wrong way is gone or impossible.

5.3 The three "one description" systems

Three places in the engine define something once and generate everything else from it. Each replaced two or three hand-maintained copies that had already drifted. If you take one architectural idea from this document, take this one.

Component reflection

Each component is described once — name, fields, types, editor hints:

```
// Order is the serialization order and the inspector order, so it is
// fixed here rather than emerging from a hash map.
ComponentDesc desc;
desc.Name = "ParticleEmitterComponent";
desc.DisplayName = "Particle Emitter";
desc.Fields = {
    Field<ParticleEmitterComponent::Rate>("Rate", Drag(1.0f, 0.0f, 10000.0f)),
    Field<ParticleEmitterComponent::ColorStart>("ColorStart", Color()),
    Field<ParticleEmitterComponent::SizeCurve>("SizeCurve",
        Named("Size curve", AssetRef(AssetType::Curve, "..."))),
    Field<ParticleEmitterComponent::Blend>("Blend", Enum(kParticleBlendNames)),
};
Bind<ParticleEmitterComponent>(desc);
```

From that one description the engine derives **four** things:

1. **The scene file format** — the serializer walks the description rather than naming fields.

2. **The inspector UI** — each field type maps to a widget, each hint refines it.
3. **Undo** — a field edit is a command carrying component name, field name, old and new value, all of which the description supplies.
4. **The C# component bridge** — C# reaches components by registry name with text values, so a component gaining a field is visible from C# the moment it exists.

Consider what the alternative costs. Adding one field to one component would mean editing a serializer, an inspector, an undo case and a scripting binding — four edits, in four files, that a compiler cannot check are consistent. The failure mode is not a crash but *silence*: the field saves and does not appear, or appears and does not save. Here, adding a field is one line and everything follows.

The general principle: **if two pieces of code must agree about a set of things, generate both from one declaration of that set.** This is the same idea as §7.9's resolved ramp — a component that cannot express a disagreement cannot have one.

The cost, and how to keep it

Type-erasure is what makes this work — fields are read and written through `void*` with a type tag — and the cost is that type errors become runtime errors. Where such an error is silent and severe, it is worth converting back into a compile error:

```
else if constexpr (std::is_enum_v<T>)
{
    static_assert(sizeof(T) == sizeof(int),
        "A registered enum field must be int-sized: reflection reads and "
        "writes it through an int*, so a narrower underlying type is a "
        "silent memory stomp. Remove the ': uint8_t' from this enum.");
    return FieldType::Enum;
}
```

An `enum class : uint8_t` compiled, serialized, drew correctly, and wrote three bytes past the end of its field. It is now a build error naming the offending enum. Note the shape of the fix: the assert lives in the one place every registration path routes through, so there is no way to register an enum and miss it.

The RHI

One interface, two implementations, discussed at length in §7.1. The same principle: the renderer describes what it wants once, and two backends derive from it.

Asset handles

One 64-bit identity per asset, one registry mapping identity to path, one `.meta` sidecar keeping the identity stable across renames. Every reference anywhere — scene files, prefabs, components, the C# bridge, the content browser — is that handle.

The payoff is best shown by something that happened late. When particle curves were added as a new asset type, **the inspector, the serializer and the C# bridge all worked immediately**, with no changes: a curve reference is an `AssetHandle`, and every one of those systems already knew what those were. A design that makes a new feature *free* in three subsystems has earned its keep.

6 The frame

What happens, in order, between one image appearing and the next. This is the chapter to read when something is drawn at the wrong time, in the wrong space, or one step behind where it should be.

6.1 Fixed simulation, free rendering

```
clamp frame time (0.25s)
InputMap::Update()           once per frame
for each fixed step:         zero or more
    Layer::OnFixedUpdate(dt)  scripts, then physics
    InputMap::EndFixedStep()
alpha = accumulator / dt
BeginFrame -> Layer::OnUpdate(ts) -> ImGui -> EndFrame
```

The maths is §2.13. What matters architecturally is the consequence: **every system must decide whether it is simulation or presentation, and the decision is not negotiable after the fact.**

The rule

Ask one question of any system: *does anything depend on this being reproducible?* If a script scores it, physics collides with it, or a replay must reproduce it, it is simulation and belongs on the fixed step. If it exists only to be looked at, it is presentation and belongs on the frame — because pinning it to the simulation rate makes it stutter at every other display rate.

`Scene::OnUpdateRuntime` is the per-frame pass, and every entry carries its reason:

```
// On the frame rather than the fixed step, for the same reason the audio
// positions are: an animation is presentation, and pinning it to the
// simulation rate would make it stutter at any other frame rate.
UpdateAnimators(ts);

// Per frame, not per step: this is where the blend between the last two
// simulation states is applied, and it is the frame that needs it.
if (m_Physics)
    m_Physics->SyncTransforms(*this, Application::GetInterpolationAlpha());

// After the sync, so a sound on a simulated body is placed where it is
// being drawn rather than one step behind it.
UpdateWorldTransforms();
UpdateAudio();

// Presentation too: nothing collides with a particle and nothing scores
// one, so they move at whatever rate the display shows them.
Particles::System::Update(*this, ts.GetSeconds());
```

Read the ordering as well as the choices. Audio comes *after* transform sync, because a positional sound attached to a simulated body should be placed where the body is being *drawn*, not where it was one step ago. That is a one-line consequence of taking the distinction seriously.

The gap this analysis exposes

Scripts are the one system still pinned to the fixed step, and applying the rule above shows why that is now a defect rather than a design. A camera moved from a fixed-step callback updates on one frame in four at 240

Hz — against a world that interpolates on all four. That is not merely unsmoothed; it is **differentially** unsmoothed, which reads worse than if nothing were interpolated at all. And no script can currently even see the alpha.

The fix is a per-frame script hook, and it is scheduled. Two decisions come with it. **The name:** `OnUpdate` already means fixed step in both scripting manuals and every existing script, so it cannot be reused — something that says frame makes the varying `dt` obvious at the call site. **The position:** after `UpdateWorldTransforms` and before `UpdateAudio`, so a camera script reads interpolated transforms rather than last step's.

The risk is misuse, not implementation. Gameplay in a per-frame hook reintroduces frame-rate-dependent behaviour — precisely what the fixed step exists to prevent. That is Unity's perennial `Update / FixedUpdate` confusion, and the mitigation is naming and documentation rather than code.

6.2 The render path, pass by pass

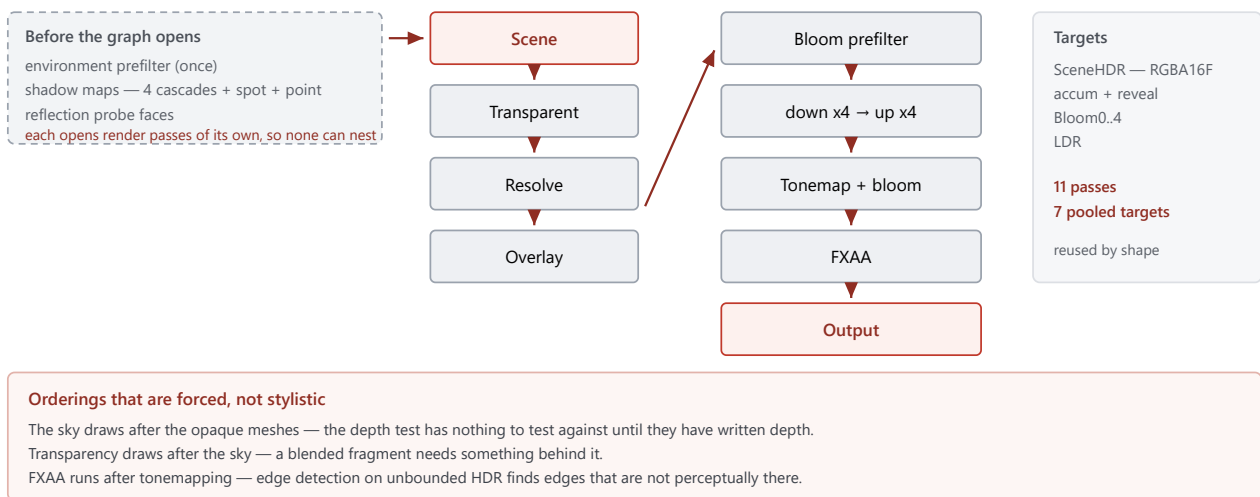


Diagram 4. One frame. The transparency passes exist only in frames whose scene contains a weighted emitter, and cost nothing in frames that do not.

Eleven passes over seven pooled targets at 1600×900. Several orderings here are forced, and each is a bug if reversed:

- **The sky after the opaque meshes**, depth-tested with no depth write. Drawing it first wastes fill on every pixel the scene will cover, and the depth test has nothing to test against until the meshes have written depth.
- **Transparency after the sky**, because a blended fragment needs something behind it to blend with.
- **The overlay into the HDR target** rather than over the finished image, because collider wireframes depth-test against the scene they annotate.
- **FXAA after tonemapping** (§7.7): edge detection on unbounded HDR finds edges that are not perceptually there.
- **Shadow maps and probe captures before the graph opens**, because each opens render passes of its own and a pass cannot nest inside another.

One non-obvious parameter is worth stating, because getting it backwards breaks every mesh pipeline at once: `Renderer::SetTargetFormats` is told what the *scene pass* writes (RGBA16F + depth), not what the viewport texture is. Pipelines are built against the attachment formats they will actually render into, and the editor's viewport texture is a different thing further down the chain.

6.3 The render graph

The problem it solves

Without a graph, a frame is a hand-written sequence: create these targets, bind this one, draw, bind that one, draw, remember to transition the first one, remember to reuse the second because they are the same size. It works at three passes. At eleven it is a web of assumptions no one dares reorder, and every new effect is surgery.

A **render graph** inverts the control. Passes *declare* what they write and what they sample; the graph allocates targets, pools them by shape, orders the passes and executes. A pass is two lambdas — one that declares, one that draws:

```
graph.AddPass("Scene",
    [&](RGPassBuilder& builder)
    {
        // Attachment 0 only: every pipeline that draws the scene declares
        // one colour format, and a pass binding three would require all
        // of them to declare three.
        builder.WriteAttachments(sceneHDR, { { 0, desc.ClearColor } });
        builder.SetClearColor(desc.ClearColor);
    },
    [draw = desc.DrawScene](RGPassContext& context)
    {
        if (draw)
            draw(context);
    });
```

What the declaration buys

- **Pooling.** Targets are reused when their shape matches, so eleven passes need seven targets. Nobody has to notice that the bloom chain can share with something else.
- **Refusal of malformed frames.** A pass binding an attachment its target does not have, or a target created and never written, are *compile errors* for the frame rather than an undefined picture. That is the graph paying for itself: these are exactly the mistakes that otherwise produce a black screen and no message.
- **Conditional structure.** The transparency passes exist only in frames whose scene contains a weighted emitter — and cost nothing in frames that do not.

Ask the scene, not the renderer. Whether to allocate the transparency attachments is decided by asking the *scene* whether it contains a weighted emitter, before the frame is described. Asking the *renderer* what it collected would answer for the *previous* frame — a one-frame lag and a dropped first frame. Any conditional structure in a graph has this hazard: the condition must be evaluable before the work that would answer it.

The payoff, concretely

Weighted-blended transparency needed a target with three colour attachments over one shared depth image, with the scene pass binding attachment 0 and the transparent pass binding 1 and 2. That is a structural

change to the frame — and it amounted to **two new graph capabilities and two pass declarations**, touching no existing pass. Compare that to what the same change would cost in a hand-written frame, where every target and transition is stated explicitly at each site.

The graph was written when the frame had three passes, which felt premature at the time. It was not: the entire argument of §4.1 is that this is one of the two places where abstraction must precede the features that would otherwise hard-code themselves.

7 Rendering engineering

7.1 Designing an RHI over two very different APIs

What an RHI is, and why you cannot skip it

A **Rendering Hardware Interface** is the layer that lets the rest of the engine say "draw this with that pipeline" without naming Vulkan or OpenGL. Even a single-backend engine benefits — but the reason to build it *first* (§4.5) is sharper than that: **you cannot design it correctly from one API**. Whatever API you have will look like the natural shape of the problem, and you will abstract its incidental details as though they were universal. Only a second, disagreeing API reveals which parts were essential.

The central question is what *level* to abstract at, and there are three answers, two of which fail:

Level	Looks like	Why it fails
Too low	A thin wrapper over one API's calls	Leaks. The other backend ends up emulating concepts it does not have — you write a descriptor-set emulator for OpenGL, and it is slower and buggier than the state calls it replaced.
Too high	<code>DrawMesh(mesh, material, transform)</code>	Cannot express anything new. The first time you need a compute dispatch or a second colour attachment, the abstraction has no vocabulary and you punch a hole through it.
The nouns	Device, buffer, texture, pipeline, shader, resource set, command list	These exist in both worlds even where only one names them. This is the level that works.

What to abstract

Objects with a lifetime and an identity: `RHIDevice`, `RHIBuffer`, `RHITexture`, `RHIPipeline`, `RHIShader`, `RHIResourceSet`, `RHISampler`, `RHICommandList`. OpenGL has no pipeline *object* — but it has pipeline *state*, and an object that records the state and applies it on bind is a faithful implementation rather than an emulation. That is the test for whether an abstraction is at the right level: **can each backend implement it honestly, or must one of them pretend?**

`RHIResourceSet` is the interesting case. Vulkan wants descriptor sets — pre-baked bundles of bindings. OpenGL wants individual bind calls before the draw. The abstraction is "a group of bindings you prepare and then bind as a unit", which Vulkan implements as an actual descriptor set and OpenGL implements as a remembered list it replays. Neither pretends.

What to expose rather than hide

Anything whose cost differs by an order of magnitude, and anything that changes what correct code looks like. **Frames in flight** is the example, and it is worth being concrete about why hiding it fails.

It is a Vulkan concept; OpenGL's driver handles the same problem invisibly. Hiding it would mean one of two things: either the engine pays Vulkan-style synchronisation costs everywhere, or it writes per-frame buffers without knowing whether the GPU is still reading them. Exposing it means the engine is written once, correctly, against the stricter model — and OpenGL implements it honestly. That decision found a real bug, §7.2 below.

What to design at the level of intent

Sometimes neither API's vocabulary translates, and the abstraction must describe the *purpose* rather than the mechanism. Buffer barriers are the clearest case. Vulkan wants an access mask and a pipeline stage; OpenGL wants a bit in `glMemoryBarrier`. There is no mapping between those vocabularies. So the RHI takes a `BufferSync` describing the **use**:

```
cmd->BufferBarrier(buffer, BufferSync::ComputeWrite, BufferSync::VertexRead);
```

"It was written by compute; it will be read as vertex data." Both backends can turn that into their own vocabulary without either of them lying, and — importantly — a reader of the engine code can tell what is happening without knowing either API.

One operation that earns its cost

```
// Records a one-shot command list and blocks until the GPU has finished it.  
// Outside the frame loop entirely: no swapchain image is acquired and  
// nothing is presented.  
//  
// It stalls, by construction -- that is what makes it usable for work whose  
// result is needed immediately, and what makes it wrong on the frame path.  
virtual void ExecuteImmediate(const std::function<void(RHICommandList&)>& record) = 0;
```

Asset upload uses it. One-off compute dispatches use it. And the reason it earns its place in an otherwise minimal interface: **it is what lets a headless test suite exercise real GPU work** with no frame loop to borrow. An abstraction that makes the engine testable is worth more than one that makes it slightly faster.

How you know the abstraction was right

Not by inspection — by a fact. **Adding the second backend touched no renderer file.** Everything above the RHI was written against the interface, so the second implementation was additive. If adding a backend requires editing the renderer, the boundary is in the wrong place, and you will find out at the worst possible time.

7.2 Frames in flight, and everything that follows

The timeline

The GPU runs behind the CPU. If the CPU submitted a frame and waited for it, both would idle half the time. Instead the CPU records frame $N+1$ while the GPU executes frame N :

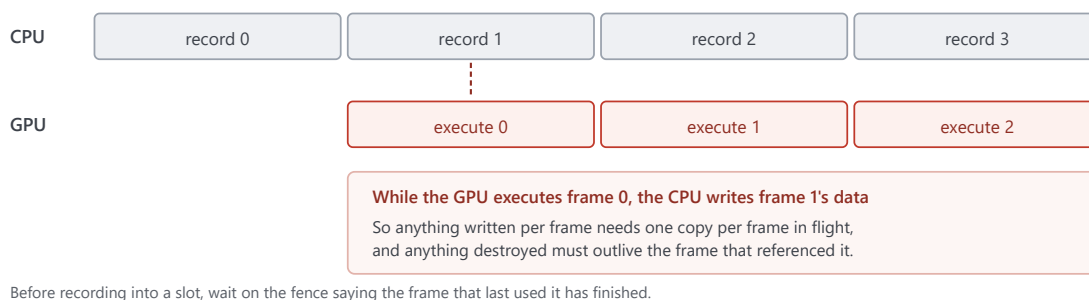


Diagram 5. Frames in flight. Every rule in this section is derivable from this one picture.

That single picture generates every rule below. **Anything the CPU writes per frame may still be in use by the GPU**, so it needs one copy per frame in flight. Anything destroyed may still be referenced by submitted work, so destruction must be deferred. And the CPU must eventually wait, or it will run arbitrarily far ahead and queue unbounded work — which is what a **fence** is for: before recording into slot i , wait for the fence that says the frame that last used slot i has finished.

Rule one: every backend needs this, including the one that hides it

OpenGL originally reported one frame in flight and had no fence. Subsystems therefore rewrote per-frame buffers the GPU was still reading, and a completely static scene rendered *three different images in six frames*. The fix — genuinely implementing multiple frames and a fence sync object — cost 6% of frame time and bought correctness. The lesson generalises past graphics: when one implementation of an interface has a constraint the other lacks, implement the constraint in both. The alternative is code that is correct in one configuration by accident.

Rule two: per-frame resources means per-frame *descriptors* too

A resource set holds one descriptor set per frame in flight, and Commit writes only the current frame's. Committing once at creation populates one and leaves the others never written. Validation catches it on frame two; a release build renders garbage. Rebind and commit every frame — it is cheap, and the alternative is a bug that appears only in the configuration where nobody is looking.

Rule three: deferred deletion, and the slot it goes in

Destroying a GPU resource the moment its last C++ reference dies is wrong: a submitted command list may still name it. So deletions go on a per-frame-slot queue and run when that slot's fence has passed. Straightforward — and the engine still got it wrong, in a way that took a purpose-built repro to find.

Entity destruction happens during the *simulation* phase, which runs **before** `BeginFrame`. A deleter pushed there was slotted by the current frame index — which is the slot `BeginFrame` was about to flush, in the same iteration. Deferred by zero frames, in every process:

```
void Push(std::function<void()> deleter)
{
    if (!DeviceAlive || PerFrame.empty()) return;

    // Out-of-frame pushes are slotted by the most recently *submitted*
    // frame, not the current index -- otherwise they land in the slot
    // about to be flushed, behind a fence covering a frame from two
    // submissions ago.
    const uint32_t count = (uint32_t)PerFrame.size();
    const uint32_t slot = InFrame ? FrameIndex : (FrameIndex + count - 1) % count;
    PerFrame[slot].push_back(std::move(deleter));
}
```

The GPU usually won the race, so it read as a rare flake — the worst kind of bug, because it resists the debugger and rewards guessing. What made it tractable was **building a repro before attempting a fix**: a scene destroying ~180 materials a second under GPU load turned it into 10 validation errors per 4000 frames, reliably. Only then could "fixed" mean something: zero in 12,000.

The general rule. Ask of every GPU resource: *who might still be reading this, and how would I know they had finished?* If the answer involves the word "probably", it is a lifetime bug waiting for a slower machine.

7.3 Batching and instancing

The cost model

A draw call is cheap on the GPU and expensive on the CPU: validating state, translating it into hardware commands, and the driver's own bookkeeping. So the cost of a frame is driven less by triangles than by **how many times you change what you are drawing with**. Two thousand identical crates should not cost two thousand of anything.

Instancing is the answer: one draw call, one mesh, N sets of per-instance data. The engine puts per-instance transforms in a storage buffer and the vertex shader indexes it by the instance index. Meshes sharing a mesh handle, material and pipeline state are collected into one such call.

The batch key, and a bug that made batching silently not happen

Grouping requires deciding when two draws are "the same", which means a key of everything that must match. The subtle failure is that *logically* identical state can be *physically* different:

Identical state must be the same object, or a batch key cannot see it. Every material created its own sampler with identical settings. The batcher compared sampler pointers, saw differences that did not exist, and silently prevented all lit-pass batching. Nothing was broken; it was merely slow, which is why it survived. Sharing one sampler took the frame to 60 draws and won 4.5× on OpenGL.

Generalise it: **a cache or a key over pointers is a key over identity, not over value**. If you intend value semantics, either intern the objects so identical values share an object, or compare the values.

The instance-index trap

A batch's base instance is a push constant, never `firstInstance`. Vulkan's `gl_InstanceIndex` includes the base offset the draw was issued with; OpenGL's `gl_InstanceID` does not. Use the API's own base-instance parameter and one backend indexes the buffer correctly while the other reads from offset zero — a correct picture on one and scrambled transforms on the other. Passing the base explicitly costs nothing and removes the disagreement.

The measurement lesson, which is the real content of this section

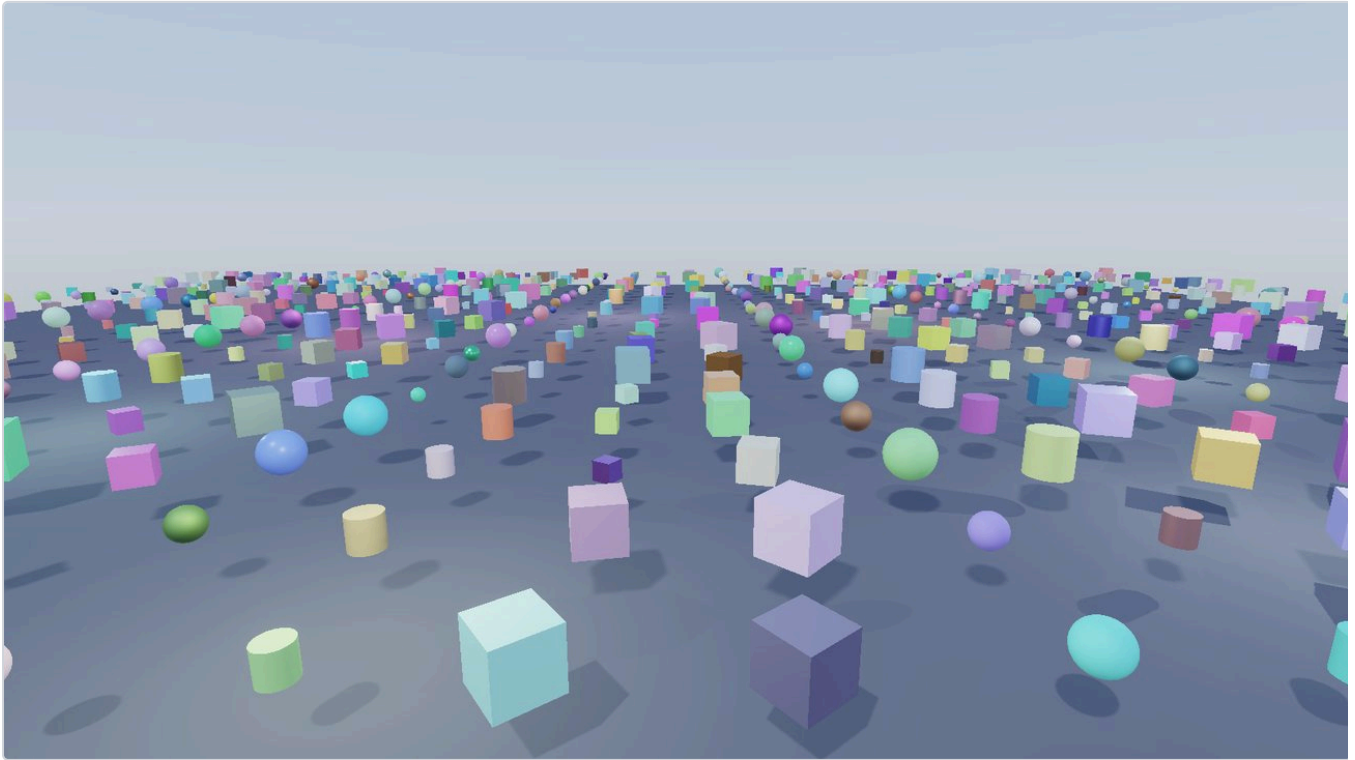


Figure 4. The stress scene — 1000 meshes at 1.88 ms (Vulkan) and 1.59 ms (OpenGL), Release, vsync off. OpenGL winning here is real and not a bug: at this draw count the frame is dominated by per-draw CPU cost, where GL's simpler binding model has less to do.

Both of this phase's optimisations produced dramatic draw-count reductions and **no frame-time improvement at all**. Frustum culling took 144 draws to 60. Instancing took 3238 to 854. The clock did not move either time.

The second one was only caught because the clock was read alongside the draw count — and reading it is what exposed the shared-sampler bug above, which was the *actual* problem. The optimisation was real; it had been silently disabled by an unrelated detail.

A draw count is not a frame time. It is a proxy, and a proxy is only useful while it correlates. Whenever you optimise, measure the thing you actually care about, in the same session, on the same machine — see §10 for why "the same session" is part of that sentence.

7.4 Lighting: clustered forward

The problem, stated plainly

Shading a pixel needs to know which lights reach it. The naive answer — loop over every light in the scene, for every pixel — costs $\text{pixels} \times \text{lights}$, and at 1600×900 with 256 lights that is 368 million lighting evaluations per frame, the overwhelming majority of which contribute nothing because the light is fifty metres away.

Three families of solution exist, and the choice constrains the rest of the renderer:

Approach	Idea	Cost you accept
Forward	Loop all lights per pixel	Quadratic. Fine to about a dozen lights.
Deferred	Write material properties to a G-buffer, then shade once per pixel per light	Large bandwidth cost, awkward with transparency and MSAA, and every material must fit the G-buffer's fixed layout
Clustered forward	Divide the frustum into cells, bin lights into cells, shade forward but loop only over the cell's lights	A binning step and an indirection, and you keep forward's freedom

This engine uses clustered forward, and the reason is the last column. Deferred would have forced a fixed material model, made the transparency work of §7.8 far harder, and spent bandwidth the engine would rather spend elsewhere. Clustered keeps forward's properties and removes its quadratic term.

The grid

The view frustum is cut into a $16 \times 9 \times 24$ grid: 16×9 tiles across the screen, and 24 **slices** along the view axis, for 3,456 cells. Each cell holds a list of the lights whose volume of influence intersects it. Binning happens on the CPU once per frame; the shader looks up one cell.

Two details in that sentence are load-bearing.

Tiles are cut in normalised device coordinates, not in world space, so a tile is a frustum-shaped wedge that widens with distance — which is exactly the shape a screen tile projects to.

Slices are cut along the camera's forward axis, not by distance to the camera. Using distance would make each slice boundary a sphere rather than a plane, and a light straddling a boundary would be binned inconsistently with how the shader looks it up. The shader is explicit about this:

```
// Distance along the camera's forward axis, which is what the slices were
// cut against -- not distance to the camera, which would make the slice
// boundaries spheres instead of planes.
float viewDepth = dot(worldPos - u_Scene.CameraPosition.xyz,
                      u_Scene.CameraForward.xyz);
```

Why the slices are exponential, and where the log comes from

If the 24 slices were spread evenly from near to far, almost all of them would land in the distant half of the frustum, where §2.5 showed there is very little going on perceptually and where a slice is enormous. Near the camera — where cells are small and lights overlap most — you would have almost no subdivision at all.

So slices are spaced **geometrically**: each is a constant *ratio* deeper than the last, which makes them uniform in log space. Given a near distance n , a far distance f and k slices, we want slice i to begin at:

$$d_i = n \cdot (f/n)^{i/k}$$

Inverting that to get a slice index from a depth gives a logarithm:

$$i = k \cdot \log(d/n) / \log(f/n) = \log(d) \cdot \text{scale} + \text{bias}$$

$$\text{scale} = k / \log(f/n) \quad \text{bias} = -k \cdot \log(n) / \log(f/n)$$

Folding the constants into a scale and a bias on the CPU turns the per-pixel work into one logarithm and one multiply-add — no search, no loop:

```
// The slice mapping is the inverse of the geometric series the grid was
// built with -- scale and bias arrive already folded, so this is a log and
// a multiply-add rather than a search.
float slice = 0.0;
if (viewDepth > u_Scene.ClusterDepth.x)
    slice = log(viewDepth) * u_Scene.ClusterDepth.z + u_Scene.ClusterDepth.w;

uint z = uint(clamp(slice, 0.0, sliceCount - 1.0));
return (z * uint(tileCount.y) + tile.y) * uint(tileCount.x) + tile.x;
```

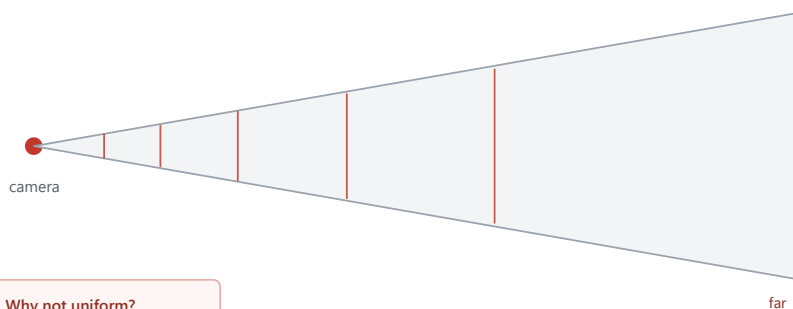
The CPU's binning and the GPU's lookup are two implementations of one mapping, and they must agree exactly. Where they disagree a fragment reads a neighbouring cell's light list — which does not look like an indexing bug. It looks like lighting that snaps as the camera moves, and you will go looking in the wrong place.

One more subtlety in that function, and it is a portability point rather than a maths one. The tile is derived from the fragment's own interpolated clip position rather than from `gl_FragCoord`:

```
// The fragment's own normalised device coordinate, from the interpolated
// clip position. Independent of the target's size and of which corner the
// backend calls the origin, both of which gl_FragCoord would drag in.
vec2 ndc = v_ClipPos.xy / max(abs(v_ClipPos.w), 1e-6) * sign(v_ClipPos.w);
vec2 unit = clamp(ndc * 0.5 + 0.5, vec2(0.0), vec2(0.9999));
uvec2 tile = uvec2(unit * tileCount);
```

`gl_FragCoord` is in pixels and measured from whichever corner the backend calls the origin (§3.4). Using it would have made the cluster lookup depend on the render target's size and disagree between Vulkan and OpenGL — the second of which produces a correct picture on one backend and misplaced lighting on the other, with no error anywhere.

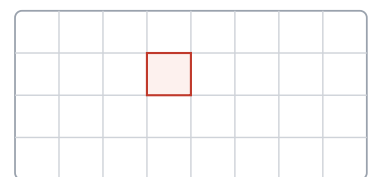
The frustum from the side, sliced along the view axis



Why not uniform?

Even spacing puts almost every slice far away, where nothing needs the detail.

slices are geometric: each is a constant ratio deeper than the last, so they are uniform in log space



16 x 9 tiles in NDC, x 24 slices

= 3,456 cells

each holding the lights that reach it

Diagram 6. The cluster grid. A fragment finds its cell with one logarithm and a multiply-add, because the mapping is the inverse of the geometric series the grid was built with — and the CPU's binning must use exactly the same mapping.

What it bought, and where it costs

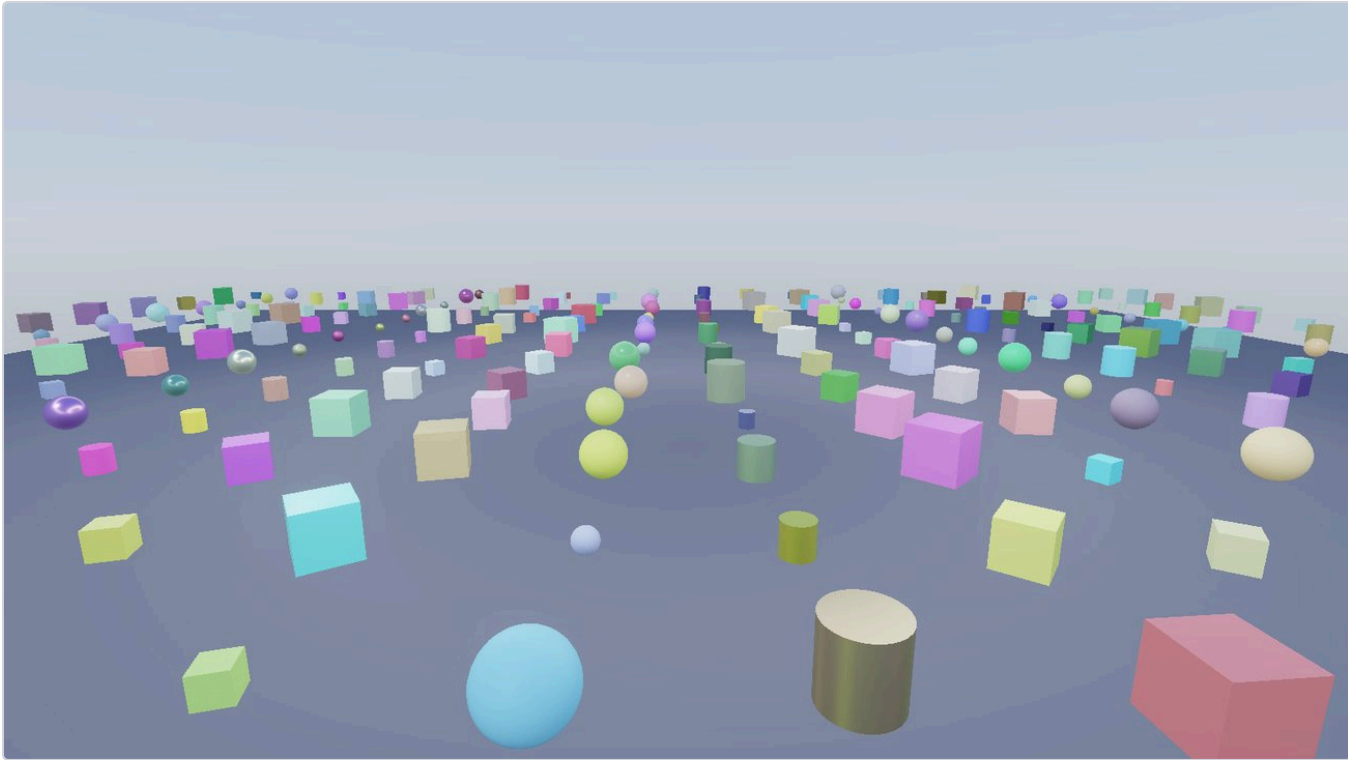


Figure 5. 256 local lights. Clustered: 3.15 ms Vulkan / 3.48 ms OpenGL, against 5.56 / 7.03 ms unclustered — with a pixel-identical image, which is the check that the optimisation is an optimisation and not a change.

Directional lights are deliberately **not** binned. A directional light reaches every cell, so putting it in every cell's list costs memory and iteration to express "always". They are stored separately and applied unconditionally.

Know where your optimisation does not help — and instrument it so you can see. When every light reaches the whole scene, the busiest cell holds all of them and clustering is pure overhead: measured at $5.76 \rightarrow 6.62$ ms, a loss. The benchmark prints "busiest cluster holds N" precisely so this is visible rather than mysterious, and the scene generator takes `--light-reach 0` to construct the pathological case deliberately. An optimisation you cannot see failing is one you will defend after it stops working.

7.5 Shadows

The idea, and why it is only an idea

A point is in shadow if something blocks the straight line between it and the light. Testing that directly is ray tracing. The rasterisation approximation is a **shadow map**, and it is elegant: render the scene *from the light's position* storing only depth. That texture now records, for every direction the light can see, the distance to the nearest thing. To shade a point, transform it into the light's space and compare its depth against the recorded one — if the point is further away than what the light saw, something is in front of it.

$$p_{\text{light}} = P_{\text{light}} V_{\text{light}} p_{\text{world}} \rightarrow \text{divide by } w \rightarrow \text{lit if } p_{\text{light}.z} \leq \text{texture}(\text{shadowMap}, p_{\text{light}.xy})$$

Everything difficult about shadow mapping comes from that comparison being between a *continuous* value and a *quantised* one.

Acne, and why the obvious fix causes a worse problem

A shadow map texel covers an area of the surface, and stores *one* depth for it. On a surface tilted relative to the light, the true depth varies across that texel while the stored value does not. Half the pixels in the texel therefore test as further than the recorded depth and declare themselves shadowed — **shadow acne**, the characteristic stripes of a surface shadowing itself.

The obvious fix is a constant depth bias: subtract a little before comparing. It works until you find the constant that removes acne on a steep surface, notice it is large, and see that on a flat surface it has pushed the shadow away from the object casting it — **peter-panning**, an object floating above its own shadow. There is no constant that avoids both, because the required bias depends on the surface's slope relative to the light.

This engine uses two mechanisms instead, and neither is a depth bias:

```
// Two biases, because one is not enough and the reason is geometric. The
// shadow pass writes back faces, which pushes each caster's recorded depth
// behind it by its own thickness -- free, and proportional to the object.
// That does nothing for a surface nearly edge-on to the light, where one
// shadow texel spans a lot of depth, so the sample is also moved along the
// surface normal by a texel scaled by the slope. Pure depth bias is the
// option that was not taken: there is no constant that avoids both acne
// and shadows detaching from their casters.
```

Back-face rendering costs nothing and scales itself: by recording the depth of each caster's *far* side, the recorded depth is already behind the lit surface by the object's own thickness. A thin object gets a small bias and a thick one a large bias, automatically.

Normal offset handles what remains. Rather than moving the depth, move the *sample position* along the surface normal by roughly one shadow texel — pushing the query out of the problematic texel entirely. The offset must grow with the slope, and the slope term is a small piece of trigonometry:

$$\tan \theta = \sin \theta / \cos \theta = \sqrt{(1 - (n \cdot l)^2)} / (n \cdot l)$$

```
float NdotL = clamp(dot(N, L), 0.0, 1.0);
// tan of the angle to the light, clamped: at grazing incidence this runs
// away, and an offset larger than the geometry detaches the shadow.
float slope = sqrt(1.0 - NdotL * NdotL) / max(NdotL, 0.15);
float offset = u_Scene.CascadeTexel[cascade] * u_Scene.ShadowParams.y *
    (1.0 + min(slope, 4.0));

vec4 lightSpace = u_Scene.CascadeLookup[cascade] * vec4(worldPos + N * offset, 1.0);
```

Both clamps are necessary. As $n \cdot l \rightarrow 0$ the tangent goes to infinity, and an offset larger than the geometry moves the sample to somewhere unrelated — peter-panning by another route.

Soft edges: percentage-closer filtering

The comparison yields a boolean per texel, so a raw shadow edge is a hard staircase of map texels. **PCF** takes several neighbouring comparisons and averages the *results* — note, the results, not the depths; averaging depths is meaningless. The hardware helps: a depth-comparison sampler performs a 2×2 comparison and returns the filtered fraction in one fetch, so a 3×3 grid of those covers 4×4 texels for nine fetches.

```
// 3x3 of hardware 2x2 taps. Nine fetches is the point at which the edge
// stops looking like a staircase and starts looking like a penumbra.
float texel = u_Scene.ShadowParams.z;
float sum = 0.0;
for (int y = -1; y <= 1; ++y)
for (int x = -1; x <= 1; ++x)
    sum += SampleCascade(cascade, vec3(coordinate.xy + vec2(x, y) * texel, coordinate.z));

return sum / 9.0;
```

Cascades: one map cannot cover a view frustum

A directional light — the sun — has no position, so its shadow map must cover the whole visible world. Consider what that means: a 2048^2 map stretched over a 1 km view distance gives one texel per half metre, and the shadow of a chair is four texels across.

The observation that rescues it is that **you do not need uniform resolution**. Things near the camera occupy many pixels and need fine shadows; things far away occupy few and do not. So the view frustum is split by distance into four **cascades**, each rendered into its own map with its own tight projection. The near cascade covers a few metres at high effective resolution; the far one covers the rest.

Three consequences worth knowing before you build it:

- **Cascades are fitted to a camera frustum**, so they must be refitted whenever the camera changes — which is why the editor's two viewports fit their own, and why shadow rendering takes the camera it is about to be drawn with as a parameter.
- **The seam between cascades is visible** unless you blend. The engine's `CascadeFactor` is deliberately split into a per-cascade function so the band between two of them can ask both and interpolate.
- **Beyond the last cascade there is no data**, and the honest answer is *lit*. Returning "shadowed" would put the far world in darkness; the code returns 1.0 when the coordinate falls outside the recorded range.

Spot lights need only one map — they have a position and a cone, which is exactly a perspective projection. Point lights need **six**, as a cube map, because they shine in every direction; that is why a point light casting shadows is the most expensive light type by a wide margin, and why it is opt-in per light.

The three invariants that cost days if missed

- **Whatever poses the lit pass must pose the depth pass**. The shadow pass runs different shaders, and it is easy to give the skinned depth shader a different bone buffer — or none. The symptom is unmistakable once seen and baffling before: a character walks and its shadow stands still.
- **Nothing is culled in the shadow pass**. Frustum culling for the *camera* is exactly wrong here — an object behind the camera can still cast a shadow into view.
- **OpenGL needs** `glClipControl(GL_LOWER_LEFT, GL_ZERO_TO_ONE)` (§2.4). Without it the depth written is in $[-1,1]$ while the comparison assumes $[0,1]$, every comparison passes, and the scene renders uniformly lit — which looks like shadows being disabled rather than like a convention mismatch.

7.6 Image-based lighting and reflection probes

What problem it solves

Point and directional lights model light arriving from a handful of directions. Real environments light objects from *every* direction — a white wall to the left tints everything left-facing. Without this, PBR materials look

like plastic in a void: technically correct and obviously wrong, and metals look worst of all, because a metal has no diffuse term and is *entirely* reflection. A metal with nothing to reflect is black.

Image-based lighting treats an environment map as the light source: the whole integral of §2.9, evaluated against a cube map instead of a list of lights.

Why it is precomputed into exactly three things

That integral cannot be evaluated per pixel per frame. The **split-sum approximation** factors it into pieces that each depend on only one or two variables, so each can be precomputed into a lookup:

1. **The irradiance cube** — the diffuse half. For each direction, the cosine-weighted average of all incoming light over the hemisphere around it. Because the result is extremely smooth, it can be stored at very low resolution — a 32^2 cube is ample.
2. **The prefiltered environment cube** — the specular half, and the clever part. A rough surface reflects a blurred version of its surroundings, and blur is what mip levels already are. So the environment is convolved with the GGX distribution at several roughness values and each result stored in a mip: **roughness becomes a mip level**, and the hardware's trilinear filter interpolates between roughnesses for free. This engine builds six levels.
3. **The BRDF lookup table** — the remainder, a scale and a bias that depend only on roughness and $n \cdot v$. Two inputs means a 2D texture, and because it depends on nothing scene-specific it is generated once at startup — 128×128 here.

At shading time those three combine into a few texture fetches, which is the entire point of the exercise.

Reflection probes: the same machinery, dynamically

A static environment map cannot show the room an object is standing in. A **reflection probe** is an entity that renders the scene into a cube map from its own position — six scene renders — and objects near it reflect that instead of the sky. The engine picks the nearest probe whose influence radius contains the camera, falling back to the sky.

Probes may be **baked** (captured once; costs nothing thereafter, right for a room whose surroundings do not move) or **realtime** (re-captured continuously, one face per frame by default). One face per frame rather than six is a deliberate trade: six renders in one frame is a visible hitch, while one is a sixth of the cost and a sixth of a second of latency on a reflection nobody is studying closely.

Two bugs this area produced, both instructive

Mip generation belongs in the command buffer that wrote mip 0. `GenerateMips` submitted its own command buffer and waited, so a probe's mip chain was built from a mip 0 that had not been rendered yet — the probe's metal read black for six frames and then corrected itself. *A bug that heals itself reads as a warm-up*, which is why it survived; it took a per-frame luminance measurement to show a hard step at frame 7 rather than a gradual fill.

The cube-map face table is the contract between capture and sampling. Six faces means six view matrices, and the order and orientation conventions must match between the code that renders them and the code that samples them. Get one face's up-vector wrong and reflections are correct in five directions and mirrored in the sixth — which almost never points at anything you are looking at.

There is also a dependency worth internalising: **IBL is derived from the sky, so it is wrong whenever the sky is**. An early bug had the irradiance falling back to a sky that was never drawn, producing ambient light from an environment that did not exist. Precomputation creates these silent couplings, and the general rule is that anything derived must be invalidated by whatever it was derived from.

7.7 Post-processing

Why there is a post chain at all

§2.8 established that lighting must happen in linear space with unbounded values. A display takes bounded, gamma-encoded values. Post-processing is the bridge, and it is also the only place where effects that need the *whole image* can happen.

Bloom, and the three decisions inside it

Bloom is the glow around very bright things — physically, light scattering inside the eye and the lens. The implementation is a blur of the bright parts, added back. Three decisions matter.

Thresholding with a soft knee. A hard cutoff — "keep pixels above 1.0" — produces a visible edge where a gradient crosses the threshold, and makes bloom pop as brightness changes. A soft knee ramps the contribution in over a range instead.

The Karis average, which exists to kill fireflies. A single extremely bright pixel — a specular glint on a curved metal — survives downsampling and reappears as a large flickering blob, because averaging a huge value with small ones still gives a large value. Weighting each tap by $1/(1+luma)$ before averaging bounds any single sample's contribution. Without it, bloom flickers on exactly the content that most needs it.

Blur by resampling, not by a wide kernel. A blur wide enough to look like bloom would need an enormous kernel at full resolution. Instead the image is downsampled in five steps with a 13-tap filter, then upsampled back with a 3×3 tent filter, *accumulating additively into the level above at each step*. Each level contributes a different blur radius, and the sum is a wide, smooth, cheap falloff. The chain has an odd number of passes — remember that; it is about to matter.

Tonemapping

Mapping unbounded linear values into $[0,1]$. Simple $x/(1+x)$ works and looks flat; the engine uses an **ACES** filmic approximation, which has the gentle toe and shoulder of film and keeps saturated highlights from tearing toward white. Then exposure, then encode to gamma. This is the *only* place a tone curve is applied, which is what makes the invariant "the PBR shader outputs linear HDR only" enforceable.

FXAA, and why it runs last

FXAA finds edges by luminance contrast and blurs along them. It runs on the **tone-mapped** image rather than the linear one, and this is not arbitrary: contrast in unbounded HDR is not perceptual contrast. A linear-space difference between 40.0 and 41.0 is large numerically and invisible on screen; run edge detection there and it finds edges nobody can see while missing the ones they can.

The bug that has now happened twice

This is the single most instructive defect in the codebase, both for what it was and for how it came back.

```
// A fullscreen pass reads one render target and writes another, and the two
// backends do not store a rendered image the same way round.
//
// Vulkan's first framebuffer row is the top of the image; OpenGL's is the
// bottom. The RHI hides that everywhere else by giving Vulkan a negative-
// height viewport, so geometry lands in the same place on both -- but that
// fixes where a fragment *writes*, not where a texture coordinate *reads*.
//
// An even number of passes hides it, which is why this survived: with
// anti-aliasing on, the scene goes tonemap -> FXAA and comes out the right
// way up. The bloom chain has an odd number, and its contribution was being
// added upside down -- visible only once something was bright enough to
// bleed, as blobs mirrored about the middle of the image.
local.FlipY = s_Data->Device->GetBackend() == Backend::Vulkan ? 1.0f : 0.0f;
```

Read that carefully, because the mechanism generalises: an *even* number of mistakes cancels. The scene survived a whole roadmap phase because the ordinary path had two flips and the bloom path had one — and the one only became visible when the scene finally contained something bright enough to bloom.

Then it happened again, and the second time is the transferable lesson. The invariant had been written down ending "...and nothing else samples a render target." That was true when written. It stopped being true the moment the transparency resolve pass of §7.8 was written — a fullscreen pass that samples two render targets, living in the particle renderer rather than the post-processing file, so it inherited none of this. One pass is an odd number. Vulkan composited every weighted particle upside down.

A rule stated in terms of where code lives will be violated by code that lives somewhere else. The invariant is now stated in terms of behaviour: anything that samples a render target and writes another owes the flip, wherever it lives. That is a rule the resolve pass could not have slipped past.

7.8 Transparency, and why it is the hardest easy problem

Opaque geometry is order-independent because the depth buffer sorts it. Transparency is not, and everything else follows from that.

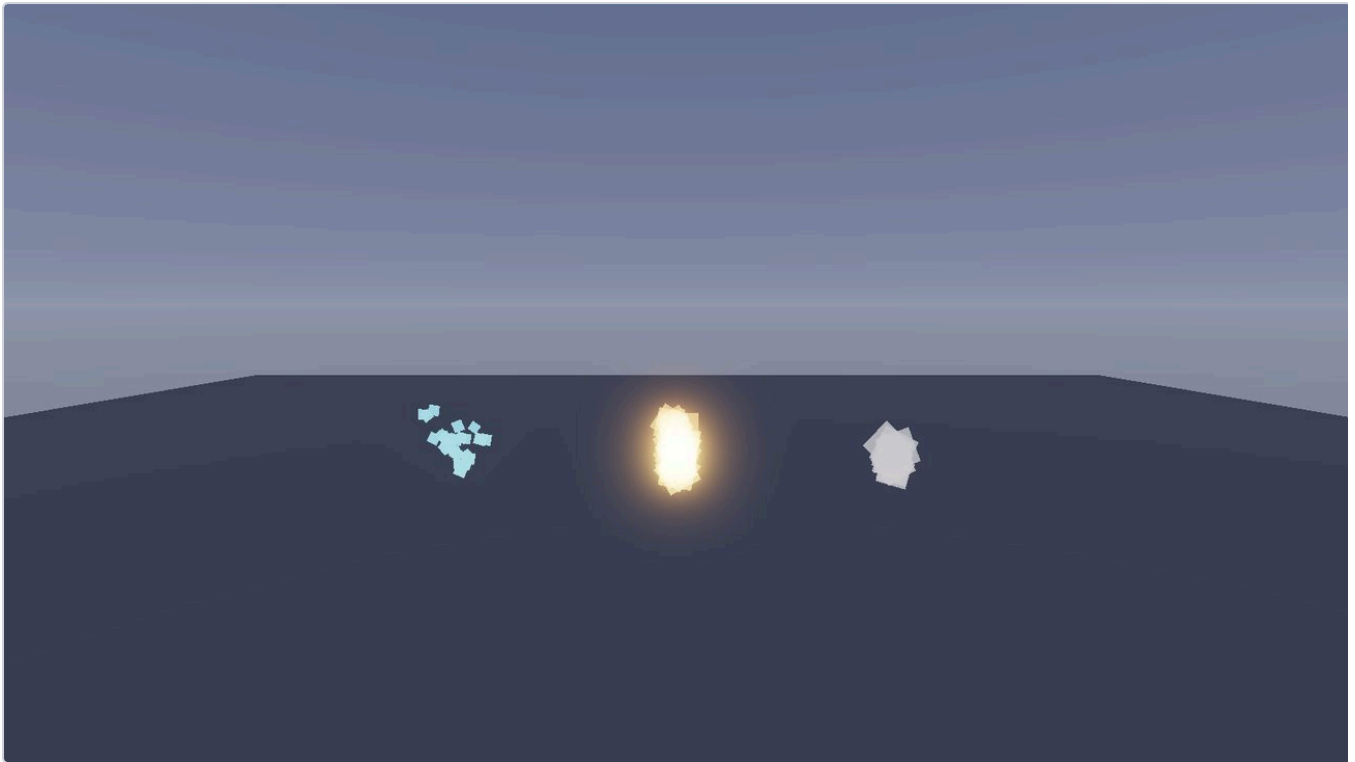


Figure 6. The three answers, in one scene. Left: a 2D flat-facing fountain. Centre: additive fire. Right: weighted-blended smoke.

Mode	How	Correct when
Additive	Sums light; order cannot affect a sum	Always — for things that <i>are</i> light: fire, sparks, flashes
Alpha	Sorted back to front	Exact, but needs a sort — impossible for a GPU-simulated pool without a readback
WeightedBlended	Two order-independent accumulators plus a resolve	Approximate, needs no sort, and the approximation is good for dense plumes

Weighted-blended OIT (McGuire & Bavoil) accumulates premultiplied colour weighted by a depth-dependent term into one buffer, and coverage into another; a resolve pass divides them. The technique lives or dies on that weight function, and validating it is a lesson in how to test a picture.

How to validate something whose failure mode is "looks fine"

The weight was taken from a well-known published snippet and eyeballed once. To check it, a scene was built whose correct answer is *arithmetic*: three emitters at 2, 20 and 200 units, one motionless particle each, red/green/blue at alpha 0.5, every spatial quantity scaled by distance so all three subtend the same angle and land exactly concentric. Compositing 0.5 red over 0.5 green over 0.5 blue is 4:2:1 — a red-dominant pixel — and a single sorted CPU emitter produces that by construction. So the reference is not a stored image but a render that is *correct by definition*.

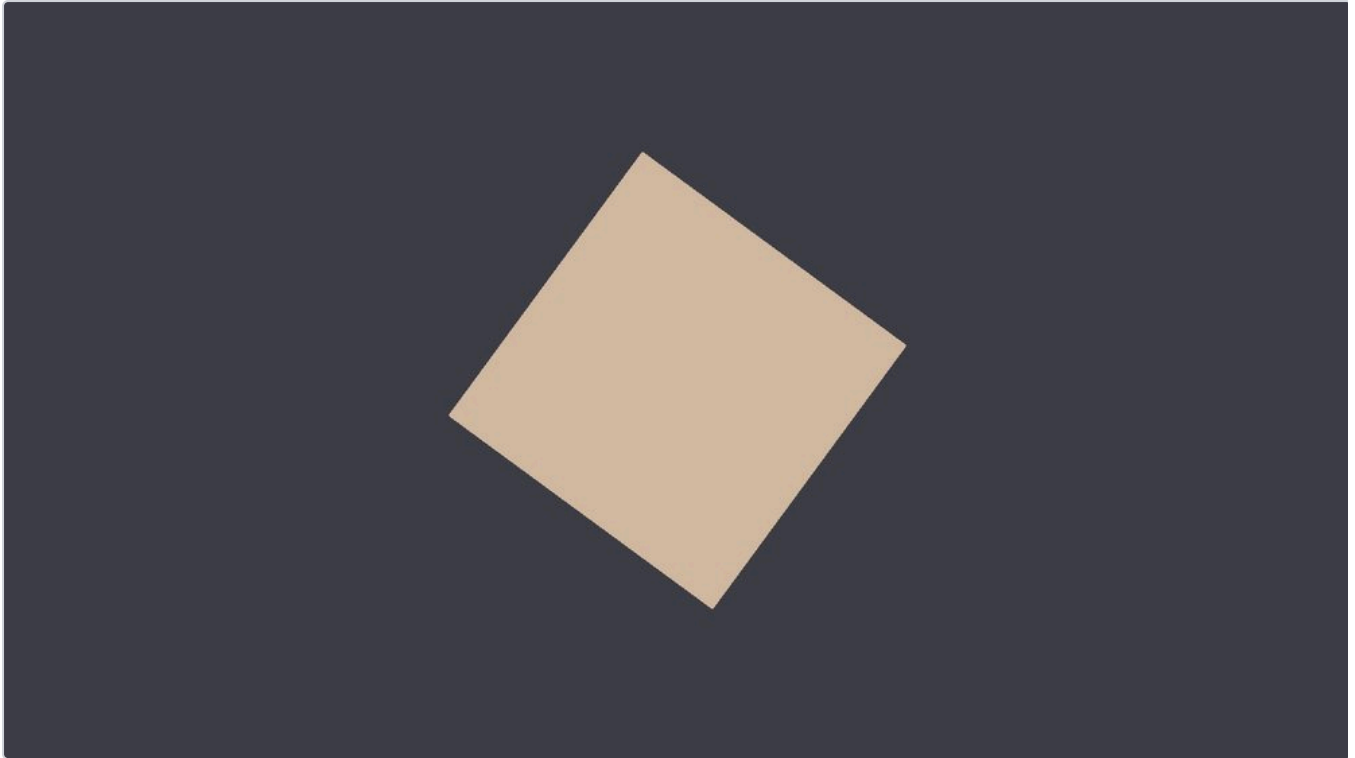


Figure 7. The instrument. Three identical emitters at 2, 20 and 200 units, drawn concentric. Rendered as sorted alpha this is the ground truth; rendered weighted it exposes exactly how much the depth weight is doing.

It found the weight was **a constant**. The published snippet uses `gl_FragCoord.z`, which is non-linear: with the sample scenes' near plane of 0.01 it spans 0.995 to 1.0 across the entire world, so the depth term varied by 4% from arm's reach to the horizon — and a `1e8` multiplier put the product 39× over the clamp ceiling at every distance anyway. Since the resolve divides accumulated colour by accumulated alpha, *a constant weight cancels exactly*: the output was an unweighted average that still looked like plausible transparency. Proven by replacing the whole expression with the literal `1.0` and getting a pixel-identical image.

A formula whose output is later normalised must be tested for whether it still varies, not whether it looks sensible. A constant is the one value that hides perfectly inside any normalisation.

The replacement falls as $1/\text{distance}$ on linear view depth. That is not the paper's recommendation, and the reason is a property the error table does not show: under the standard clamp, the paper's equation 9 only discriminates between 1.5 and 100 units and clamps flat outside that band — it draws the 20-unit and 200-unit layers identically. $1/d$ discriminates from 0.03 to 9000 units, so ordering never collapses, and the range it buys is at the *near* end, which is where a camera standing inside a smoke cloud actually lives.

7.9 Compute, and moving a simulation to the GPU

What a compute shader is

Everything so far has gone through the graphics pipeline of §3.1: vertices in, triangles rasterised, fragments out. A **compute shader** removes all of that. There is no vertex stream, no rasteriser, no render target — just a program and a number of times you would like it run. You decide the shape of the work:

```
layout(local_size_x = 64) in;    // 64 invocations per workgroup
// ...
uint index = gl_GlobalInvocationID.x;    // which item am I?
```

Invocations are grouped into **workgroups**, and a `Dispatch(n)` launches n workgroups. The grouping is not cosmetic: invocations within a workgroup can share fast on-chip memory and synchronise with each other, while invocations in different workgroups cannot do either. That distinction is what makes some algorithms (a sort, a reduction) cheap if the data fits in one workgroup and expensive if it does not — and it is why §11's proposed bitonic sort is a single dispatch for pools up to 2048.

Two arithmetic details bite immediately. The dispatch count is in *workgroups*, not items, so covering n items needs $\lceil n/64 \rceil$ groups — and the last group runs past the end, so **every compute shader needs a bounds check**. The engine derives the group count from shader reflection rather than a hard-coded 64, so that changing `local_size_x` cannot leave a dispatch covering a fraction of its data:

```
cmd->Dispatch(pipeline->GroupsFor(particleCount));    // reads local_size_x from reflection
```

Why move particles to the GPU at all

A particle simulation is the ideal candidate: thousands of items, identical work per item, no interaction between them. On the CPU it costs a loop over sixteen thousand particles *and* the bandwidth to upload the results every frame. On the GPU the data never leaves the device.

	CPU simulation	GPU simulation
Vulkan	3.30 ms	1.37 ms
OpenGL	3.80 ms	1.58 ms

2.4× on both, at ~16k particles across three emitters. More telling than the ratio: after the change both backends are GPU-bound — frame time $\approx$ GPU work — where the CPU path had roughly 2 ms of CPU sitting on top.

The design, and the constraints that shaped it

Each emitter gets a fixed-size pool in a device-local storage buffer. One dispatch per emitter ages, integrates and spawns. Three decisions follow from what the GPU can and cannot do:

- **Spawning is a ring over the pool, not an append.** Ten thousand invocations have no shared cursor to push onto. Instead each slot works out whether it falls inside this frame's spawn window, and a slot inside the window that is still alive is left alone — which reproduces "a full pool refuses to spawn" from the other direction.
- **Dead particles collapse to zero size rather than being compacted.** Compaction needs a prefix sum and a second pass; a degenerate quad costs a discarded triangle. That is the right trade until profiling says otherwise.
- **The RNG is per particle, not per emitter.** The CPU path advances an xorshift state in spawn order — but there is no order here. Each invocation instead hashes its own slot index and the frame epoch, giving different numbers with the same distribution, reproducibly.

```
// Integer hash, so a slot's numbers depend on nothing but the slot and the
// epoch. Two invocations never share a sequence, and a frame is reproducible.
uint Hash(uint value)
{
    value ^= value >> 16; value *= 0x7feb352du;
    value ^= value >> 15; value *= 0x846ca68bu;
    value ^= value >> 16; return value;
}
```

A note on where the dispatch is issued, because it is a constraint newcomers hit: **a dispatch cannot be recorded inside a render pass.** The simulation therefore runs in the per-frame scene update, before the graph opens — and that placement has a second reason worth knowing, since the editor renders more than one view:

```
// Here rather than in OnRender for two reasons, both of which are bugs in
// the version that looks more natural. This function runs once per frame;
// OnRender runs once per *view*, and the editor has two -- a simulation
// stepped there would advance at double speed with the game view open. And
// a dispatch is illegal inside a render pass.
if (RHI::RHICmdList* cmd = Renderer::GetCmdList())
    Particles::Gpu::Simulate(*this, *cmd, ts.GetSeconds());
```

Barriers: telling the GPU that one thing precedes another

The compute shader writes the instance buffer; the draw reads it as vertex data. Nothing makes that ordering automatic — you must say so. The engine brackets the whole batch of emitters with one pair of barriers rather than wrapping each dispatch, and the difference is not stylistic:

Interleaved barriers serialise the work. A barrier after every dispatch means every emitter waits for the one before it, so a dozen emitters run sequentially on hardware capable of running them together. Bracketing the batch lets them overlap. Measured as a real improvement — though notably *not* the fix for the problem below, which is the point of the next paragraph.

The performance bug, and why the first hypothesis was wrong

The first working version of the GPU path was **slower than the CPU one** — 6.9 ms against 3.3 — and GPU timestamps refused to resolve at all. The initial hypothesis was barrier interleaving. Batching the barriers helped a little and did not fix it.

The actual cause was a memory-domain choice. The particle state buffer had been made **host-visible** so that the rare "seed the pool from the CPU" path could be a `memcpy`:

A persistently-mapped host-visible buffer written by a compute shader is a synchronisation point on OpenGL. The driver must reconcile CPU-visible memory with GPU writes, and it does so by stalling. Device-local storage with a staging upload for the rare seed fixed both the speed and the missing timestamps.

Two transferable lessons. **Memory domain is a performance decision, not a convenience one** — "the CPU might want to touch this occasionally" is not a reason to make a buffer host-visible, because the cost is paid continuously and the benefit occasionally. And **a plausible hypothesis that yields a small improvement is the most dangerous kind**: it produced a real gain, which made it feel confirmed, and the actual 2× problem sat untouched behind it. Only measuring again, rather than declaring victory, found it.

Keeping two implementations of one behaviour honest

Once there are a CPU and a GPU simulation of the same thing, every future feature has to be implemented twice — and the two will drift. Curve-driven ramps (§11) were the test case. The obvious design gives the compute shader its own copy of the rule deciding whether a curve or a start/end pair governs each channel. The engine does the opposite: the CPU resolves the ramps once per emitter per frame and uploads the *answer*.

```
// The ramps, already resolved on the CPU by Particles::Evaluate.  
//  
// This shader is handed the answer rather than the ingredients: it has no  
// idea whether a channel came from a curve asset or a start/end pair, which  
// is deliberate. That rule lives in exactly one place, and a shader that  
// cannot express it cannot disagree with it.  
vec4 ColorRamp[64];  
vec4 SizeRamp[64];    // x only
```

Two kilobytes per emitter, to delete an entire class of bug. The general form: **when two implementations must agree, move the decision out of both and hand each the result**. A component that cannot express a rule cannot disagree about it.

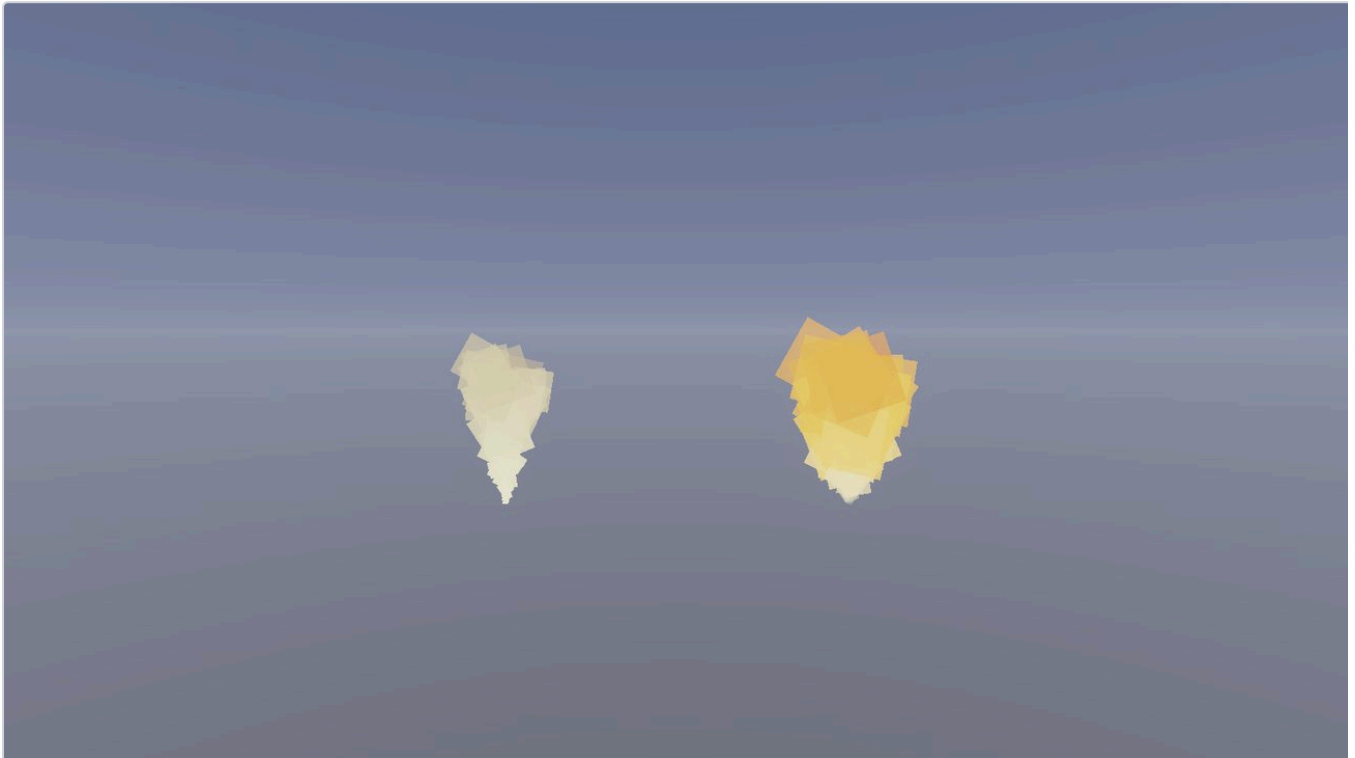


Figure 8. What the ramps buy, and why they were worth this trouble. The same emitter twice: left, size, colour and alpha as start/end pairs; right, the same three driven by curves. The gradient passes *through* orange rather than crossfading past it, the size curve rounds the plume out early, and an alpha curve fades it in *and* out — which two endpoints cannot express at all, since that needs three values and a pair has two.

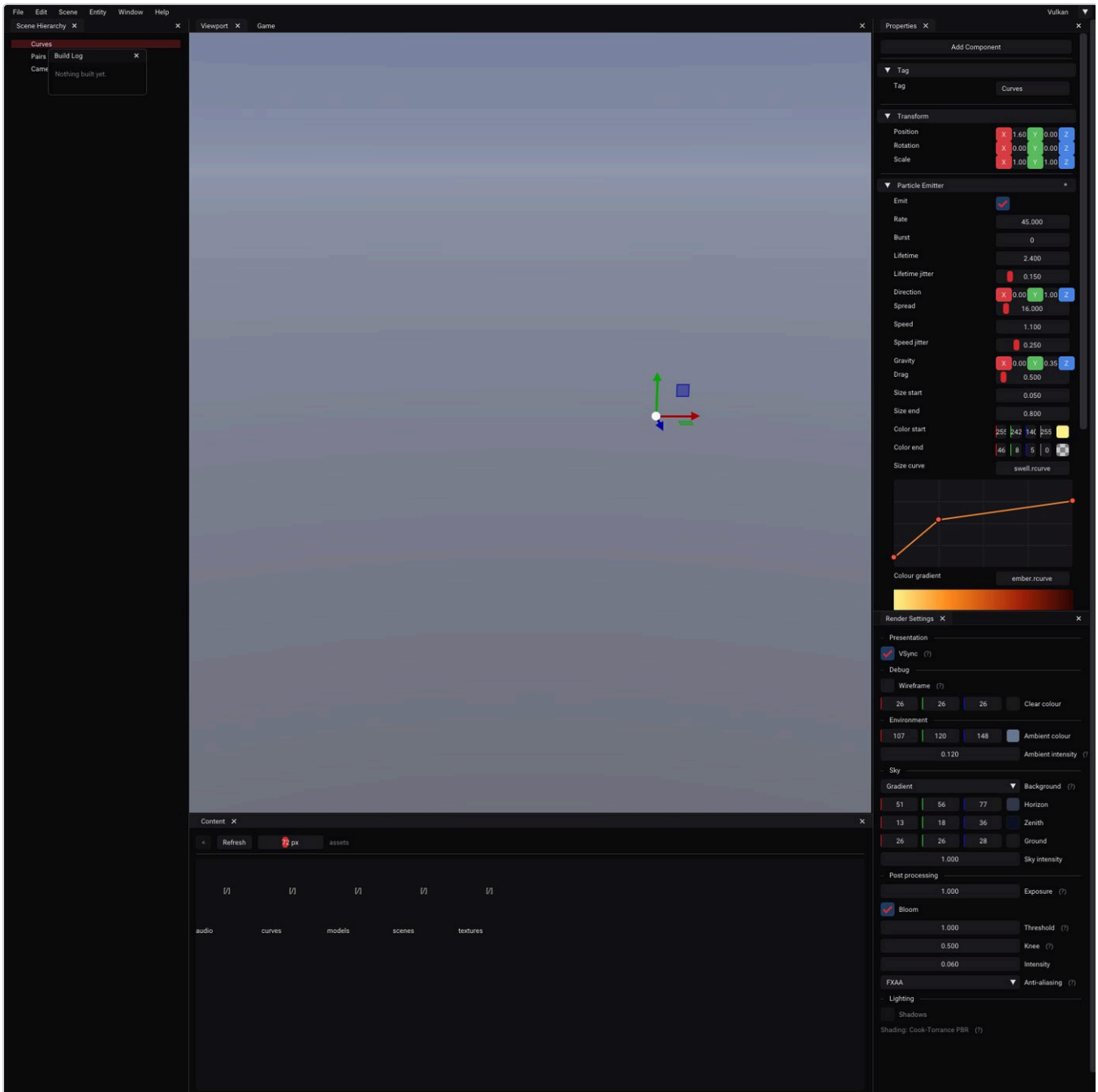


Figure 9. Curves are assets, edited inline under the field that references them — a graph with draggable points for a scalar ramp, a stop strip for a gradient. Because a curve reference is an ordinary `AssetHandle`, the inspector, the scene file format and the C# bridge supported it with no changes at all (§5.3).

What measuring the agreement then found

Verifying that took rendering the same scene both ways. Under `Additive` they agree to **0.10 of 255** on both backends. Under `Alpha` a curve-driven emitter differed by **2.7 — seven times the run-to-run noise**.

That gap is not the ramp. Switching the blend to an order-independent one collapses it to 0.10, which pins it on the GPU path's inability to sort within an emitter (§7.8) — and reveals something that was not obvious: **curves make the sorting gap worse**, because they widen how much particles differ from each other in colour and opacity, which is precisely the condition under which unsorted alpha goes wrong.

Note the shape of that investigation, because it is the reusable part. A discrepancy was found; two explanations were available; and rather than reasoning about which was more likely, **one variable was**

changed to make the explanations predict different outcomes. Order-independent blending should erase a sorting problem and leave a data problem untouched. It erased it.

8 The systems around the renderer

8.1 Scene, entities, reflection

Why an ECS, in one paragraph

The obvious way to model game objects is inheritance: a `GameObject` base, then `Enemy`, then `FlyingEnemy`. It works until you want a flying enemy that also talks, and a talking crate, and now the hierarchy has to be a lattice. An **entity-component system** replaces "what an object is" with "what an object *has*": an entity is an identity, components are plain data attached to it, and systems operate on everything that has a given combination. A talking crate is a crate with a dialogue component. No hierarchy is required and none can be got wrong.

This engine uses **EnTT** for the storage — a mature sparse-set implementation — and builds four things on top of it, each of which is the interesting part.

1. Identity that outlives the registry

EnTT's `entt::entity` is an index with a generation counter, and it is **recycled**. Delete an entity and the next one created may get the same handle. It is therefore meaningless the moment anything needs to remember an entity across a save, a reload, a play-mode copy or a network message.

So every entity also carries a 64-bit UUID assigned at creation and preserved through all of those. Every durable reference in the engine — a parent link, a prefab's contents, a script's cached target, the physics body map — is a UUID, never a handle.

This is the single highest-leverage decision in the project, and it costs an afternoon. Retrofitting identity later means touching every one of those systems simultaneously, and each of them is individually easy to get subtly wrong. Recycled handles fail in the worst possible way: correctly, until a deletion happens to recycle the exact handle something remembered, at which point one object starts behaving as another.

2. Hierarchy stored as one link, not two

A transform hierarchy needs parents and children. Storing both invites them to disagree. The engine stores only the parent, by UUID, and *rebuilds* the child lists on load:

```
// Children is derived state: only Parent is written to disk, and the child
// lists are rebuilt on load. Storing both would let them disagree, and a
// hierarchy that disagrees with itself is the kind of bug that only shows
// up three features later.
struct RelationshipComponent
{
    UUID Parent = UUID::Invalid();
    std::vector<UUID> Children;
};
```

The general form — **serialize the minimum from which everything else is derivable** — applies well beyond scene graphs. Derived state that is also persisted is state that can be persisted wrong.

3. World transforms, recomputed rather than cached

A child's world transform is its parent's world transform times its own local one. The obvious optimisation is a dirty flag: mark it when the local transform changes, recompute only what is dirty. The engine deliberately does not:

```
// Recomputed unconditionally by Scene::UpdateWorldTransforms in one
// top-down pass per frame.
//
// Deliberately not a dirty-flag cache. A flag has to be set at every write
// site -- the inspector, the gizmo, scripts, the serializer, and everything
// added later -- and a single missed one leaves an object silently
// rendering in the wrong place. Recomputing is O(n) at scene scale and
// cannot be got wrong.
Mat4 World{ 1.0f };
```

Read the argument carefully, because it is a template for a class of decision. The cache is faster. But its correctness depends on *every future write site* remembering to invalidate, including ones written by people who have never heard of the flag. The cost of one missed invalidation — an object drawn in the wrong place, intermittently — vastly exceeds the saving, and the traversal is $O(n)$ over a scene graph, not over anything large.

Choose the boring implementation when correctness depends on future discipline. Optimise when a measurement says to, and then encapsulate the invalidation so it cannot be forgotten.

4. Reflection

Covered in §5.3, and it belongs to this layer: the component registry is what turns "plain data attached to an entity" into something the serializer, inspector, undo stack and C# bridge can all handle generically.

Play mode: the scene is copied, not mutated

Pressing Play operates on a *copy*; pressing Stop discards it and restores the authored scene exactly. This is worth building before scripts exist, because it establishes a rule scripts then cannot violate: nothing a script does at runtime can damage the thing on disk. The consequence users notice is that changes made during play are discarded — correct, and surprising exactly once.

8.2 Assets

The three problems an asset system solves

They are separable, and it helps to name them:

1. **Identity** — how a scene refers to a mesh without embedding it, in a way that survives the file being renamed or moved.
2. **Loading and caching** — turning a reference into a usable object once rather than per frame.
3. **Import** — turning somebody else's format into yours.

The engine answers them with a 64-bit `AssetHandle`, a registry mapping handle $\rightarrow$ path, and a `.meta` sidecar next to each source file recording its handle. The sidecar is what makes renaming safe: the identity travels with the file rather than being its path.

Two caching conventions worth copying

Cache failures, not just successes. A missing texture caches as null. Without that, a scene pointing at a deleted file retries the load sixty times a second forever — a performance bug that presents as "the editor is slow with this scene" and takes a profiler to find.

Never return null for a valid handle where a value will do. This one is subtler and shaped an entire feature:

```
// A keyed ramp -- size or alpha over a particle's life, or the colour
// gradient. Never null for a valid handle: a missing or unreadable file
// answers an *empty* curve, which evaluates to its fallback everywhere,
// so a caller reads a value rather than remembering a branch. Cached,
// including the failure, so a broken path is not reopened per frame.
static const Curve* GetCurve(AssetHandle handle);
```

Only an *invalid* handle answers null. That distinction — "no curve was authored" versus "a curve was authored and failed to load" — is exactly what the caller needs, and it means the per-particle sampling path has no branch in it at all.

The stale-derivative problem

Any cache of something *derived* from an asset needs an invalidation path, and it is easy to build the cache and forget the path. The curve system caches both the curve and a baked 64-sample table of it; both are dropped together by one function, which the editor calls when someone finishes dragging a point. Without it, an edited curve keeps rendering with its old shape and the editor appears to ignore you.

The same class of bug, in a different subsystem, is the probe reading a mip chain built from an un-rendered mip 0 (§7.6). **Whatever is derived must be invalidated by whatever it was derived from**, and the invalidation should live next to the cache rather than at the call sites.

A trap in the build

Asset handles minted in the build output are lost on a clean build. The assets root is the folder beside the executable, which CMake copies from the source tree. Assets added to the *source* tree keep their handle because the `.meta` is copied with them; assets dropped into `build/` get a fresh handle each time the directory is recreated, and every scene referencing them breaks.

8.3 Scripting in two languages

Why two, and what that costs

C++ gives direct access and no marshallng; C# gives iteration speed, memory safety and a much shorter path from idea to running code. Supporting both is a deliberate bet that the audience wants both, and the cost is real: **every feature must be exposed twice**, and the two surfaces will drift unless something structural stops them.

C++ scripts live in a project's `Source/` and compile into a game module DLL the engine loads (§5.2). C# scripts live in `Scripts/` and compile into an assembly. Both hot reload.

Hot reload, and the part people get wrong

For C#, the assembly is loaded **from bytes** into a collectible `AssemblyLoadContext` that is retired and replaced on every build. Loading from a file path would lock the file and the next build would fail; loading from bytes leaves the file free.

The retirement is the subtle part. Unloading a context is a *request*, not a command: it succeeds only when nothing references anything in it. So the engine holds a weak reference and checks:

Verify the unload, and report a context that will not die. A leaked context means the old code is still running, and the symptom is "my change did nothing" — which the developer will attribute to the build, the editor, or themselves, in that order, for about an hour. An explicit warning turns a mystery into a message.

Building while playing is handled by one rule applied to both languages: stop the scene, swap both languages' scripts, resume play on the new code. After a *failed* build it stays stopped with the errors visible, rather than auto-resuming old code under you.

The interop boundary

The native side exposes an append-only table of function pointers with an explicit protocol version in both the native and managed headers. Append-only matters: adding a function must never move an existing one, or a mismatched pair of binaries fails in a way no error message will explain.

There is one structural asymmetry, and it is inherent rather than incidental. A template cannot cross a language boundary, so where C++ says `GetComponent<LightComponent>()`, C# reaches components **by registry name with values as text**:

```
entity.SetComponentField("LightComponent", "Intensity", "3.5");  
string color = entity.GetComponentField("LightComponent", "Color"); // "1 0.8 0.6 1"
```

That looks like a downgrade and is the reason the two surfaces do not drift. Because the bridge is driven by the same component registry as the inspector and the scene file (§5.3), **a component gaining a field is reachable from C# the moment it exists** — with no binding to write and nothing to forget. A typed binding would have needed regenerating, and the regeneration would have been skipped once.

Errors at the boundary

Every boundary the runtime crosses catches everything: an exception in a script logs and stops that script's callback, and the editor survives. That is the deliberate trade against C++, where a mistake is a crash. It has a cost worth stating in the manual: a silent `catch` can hide a broken script, so the log is the only place the truth appears.

8.4 Physics and audio

Physics: ordering within the step

Jolt provides the simulation. What the engine decides is *when* things happen relative to it, and the order is fixed and documented: **scripts, then physics, then contact events**, all within the fixed step.

Scripts first so that a force applied this step is integrated this step rather than next. Contact events after the step that produced them, so a script reacting to a collision sees the post-collision state. Any other order produces a one-step lag that is invisible in testing and infuriating in a game.

Two lessons from wiring a physics engine in

A body added during play must actually join the simulation. The engine had an `AddBody` function with no callers — bodies were created when a scene started playing and never afterwards. Everything looked complete, and the first object spawned at runtime hung motionless in the air. The fix reconciles bodies at the top of the fixed step, and it must skip entities already condemned by a deferred destroy, or destroying an entity re-adds its body moments before removing it.

Collider dimensions are multiplied by the transform's absolute scale. A sphere primitive is built at radius 0.5, so a collider for a mesh scaled 1.8 is authored as `Radius: 0.5`, not 0.9. Authoring the world radius produces a collider 1.8× too large, and objects hover above apparently nothing — a bug with no error and a very confusing picture.

Audio: presentation, and therefore per frame

Audio is miniaudio. Positional sources are updated per *frame*, after transform sync (§6.1), so a sound attached to a simulated body is placed where it is being drawn rather than one step behind. Voices are retired once a frame, unconditionally: a one-shot fired from a script belongs to nothing that would otherwise clean it up.

Two API decisions worth copying. Clips are named by **asset path** rather than by handle, because a handle is an engine-internal name a script has no honest way to hold. And every one-shot takes a **pitch**, which is a resampling ratio — 2 is an octave up and half the length. A small random spread on pitch is what stops a repeated impact sounding like a sampler, and it costs one line in a script.

Skeletal animation

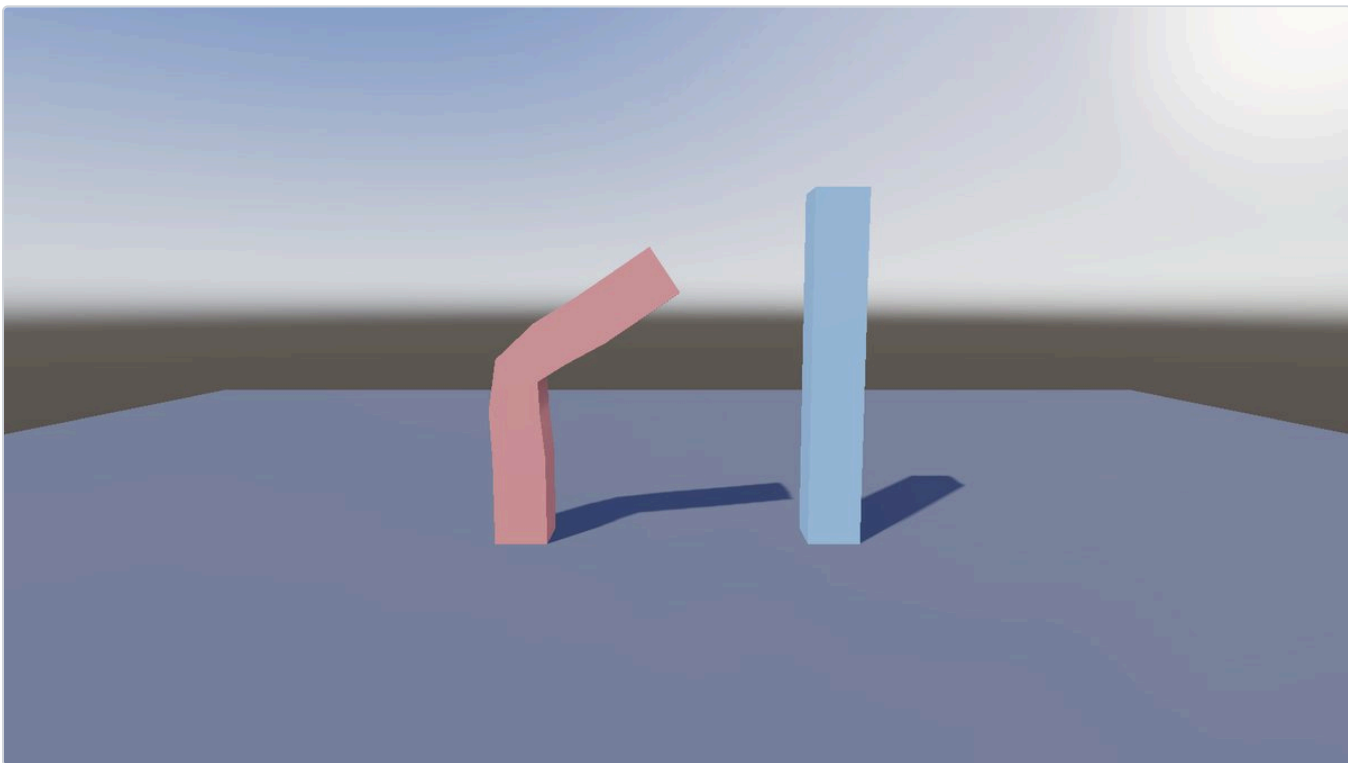


Figure 10. GPU skinning. The maths is §2.12; the engineering is keeping the bone matrices consistent between the lit pass, the shadow pass and the bounds.

Vertices carry joint indices and weights; a bone matrix palette is uploaded per frame; the vertex shader blends. Three invariants, each of which produced a memorable failure:

- **A skeleton at its bind pose must give an identity skinning matrix for every bone** (§2.12) — the single check that catches wrong inverse binds, wrong compose order and wrong space at once.
- **One bone matrix per bone, never a short run.** A truncated palette reads into the next character's matrices, which animates one character with another's motion.
- **Whatever poses the lit pass must pose the depth pass** (§7.5), or a character walks and its shadow stands still.

And one that belongs to the vertex format rather than to animation, included because it is the best example in the codebase of a silent size mismatch:

A vertex attribute's reflected format must carry its vector width. The shader reflection returned a one-component format for every integer type, so a `uvec4` of joint indices was described as 4 bytes where the buffer held 16. The mesh drew as a spray of triangles across the screen — which looks like a corrupt index buffer, and sends you to entirely the wrong place.

9 What experience taught

Every entry below was a real bug that cost real time, and most of them fail silently rather than obviously. They are grouped by the kind of mistake rather than by subsystem, because the kind is what transfers.

Things that are two things pretending to be one

- **A swapchain is retired, not destroyed and replaced.** Pass the old handle as `oldSwapchain` and destroy it after the new one exists — `vkDeviceWaitIdle` covers the queues, not the presentation engine.
- **A cache holding one object under two keys must be cleared as one.** The texture loader keyed by path and the IBL filter keyed by pointer were cleared at different times; changing project kept every old environment map alive.
- **Identical state must be the same object** (@@§9.3).

Things that are silently the wrong size

- **A vertex attribute's reflected format must carry its vector width.** The reflection returned a one-component format for every integer, so a `uvec4` of joint indices was 4 bytes where the buffer held 16, and the mesh drew as a spray of triangles.
- **A registered enum field must be int-sized** (@@@@§9.3).
- **A skinned mesh needs one bone matrix per bone, never a short run** — it reads into the next character's.
- **A descriptor pool is a chain, not a ceiling.** 1000 meshes segfaulted a fixed 2000-set pool.

Things that differ between backends and look fine on one

- **Fullscreen passes and the V flip** (@@§9.7) — and an even number of passes hides it.
- **Base instance** (@@§9.3).
- **Clip control on OpenGL** (§5.5).
- `glDrawBuffers` **is persistent framebuffer state**, and `glClearBufferfv` indexes the draw-buffer list rather than the framebuffer. Binding attachments 1 and 2 means clearing indices 0 and 1.
- `independentBlend` **is a Vulkan device feature.** Different blend state on different attachments is not free.

Things about lifetime

- **Deferred deletion must be slotted by the frame actually submitted** (@@§9.2).
- **Never rewrite a bound descriptor set.**
- `Layer` **'s destructor must stay virtual.** Without it only `~Layer` ran and every derived member leaked — the editor's scene, its render targets, every material in them. Undefined behaviour that survived months because nothing looked wrong.
- **Wait for the device to idle before destroying anything**, not partway through.

- **Verify shutdown by exiting, not by killing.** Every one of these was invisible while runs were terminated rather than closed.

Things about measurement

- **A draw count is not a frame time** (§4.5).
- **A bug that heals itself reads as a warm-up** (§5.6).
- **When a timing bug will not reproduce, failing to reproduce it is the evidence** — stop asking what you broke and go find the order-dependent hazard in that path.
- **Ask whether a defect predates your change.** Stashing and rebuilding takes ten minutes and regularly turns "my regression" into "already there".
- **Benchmark runs taken far apart are not comparable.** Re-running an unchanged baseline forty minutes later drifted 1.3% with no code involved.

10 Verification: how any of this is known to work

A renderer is uniquely hostile to testing: its output is a picture, its failures are often plausible pictures, and the usual unit-test instinct produces tests that pass while the screen is blank. This is the methodology that emerged.

The bar, in full

Every change is verified against all of the following before it is committed:

1. **888 automated checks, on both backends, exit code 0.** Not "no failures printed" — the exit code, because a harness that crashes halfway prints no failures.
2. **Zero validation-layer messages**, with validation explicitly enabled. It is off by default because it costs ~1.5 ms/frame and made Vulkan read as slower than OpenGL; a run without the flag reports zero messages whether or not there were any.
3. **The pixels, via the engine's own screenshot capture** — not a draw count.
4. **Both backends' frames compared against each other**, not merely each on its own. A Vulkan-only bloom flip survived an entire roadmap phase of clean runs because nothing in the scene was bright enough for a mirrored contribution to show.
5. **All three build configurations** for anything the editor displays.

Test the property, not the plumbing

The distinction that makes GPU testing tractable. A test that asserts a dispatch was recorded proves nothing. A test that runs 100 elements through a 64-wide workgroup — so the tail group runs past the end and the shader's bounds check is actually exercised — and then checks every element, proves the thing you care about.

Compare against something correct by construction

The strongest tool in the box, and the one that found two shipped bugs in an afternoon (@@§9.8). Do not compare a render against a stored reference image, which rots and which nobody can debug. Compare it against *another render* whose correctness is guaranteed by construction — a sorted single emitter is exactly correct, so it is the reference for an unsorted approximation of the same particles.

Reintroduce the bug to prove the test fails

A regression check that has never failed is a hypothesis. Both transparency bugs were deliberately put back to confirm the new check catches them and names them correctly. The same discipline was applied to the enum compile-time assert.

Build the repro before the fix

The deferred-deletion race was a rare flake until a scene was built to destroy ~180 materials a second under GPU load. Then it was 10 errors in 4000 frames, reliably — and therefore provably fixed at zero in 12000.

Automate the eyeball where you can

Some checks genuinely need a picture, but "a picture" can still be reduced to a number. The transparency regression check renders the same particles two ways and compares *silhouettes* and channel ordering, which catches both a vertical flip and a weight that has stopped discriminating — neither of which a human reliably notices in a still of smoke.

11 Where it stands, and what comes next

Complete

Phases 0 through 5, the game-module arc, and a shipped mini game. The renderer, scene, asset, scripting, physics and audio systems described above are all in place and verified on both backends.

In progress

Phase 6. Particles are done — CPU and GPU simulation, three blend modes, and asset-backed curves for size, colour and alpha with an inline editor.

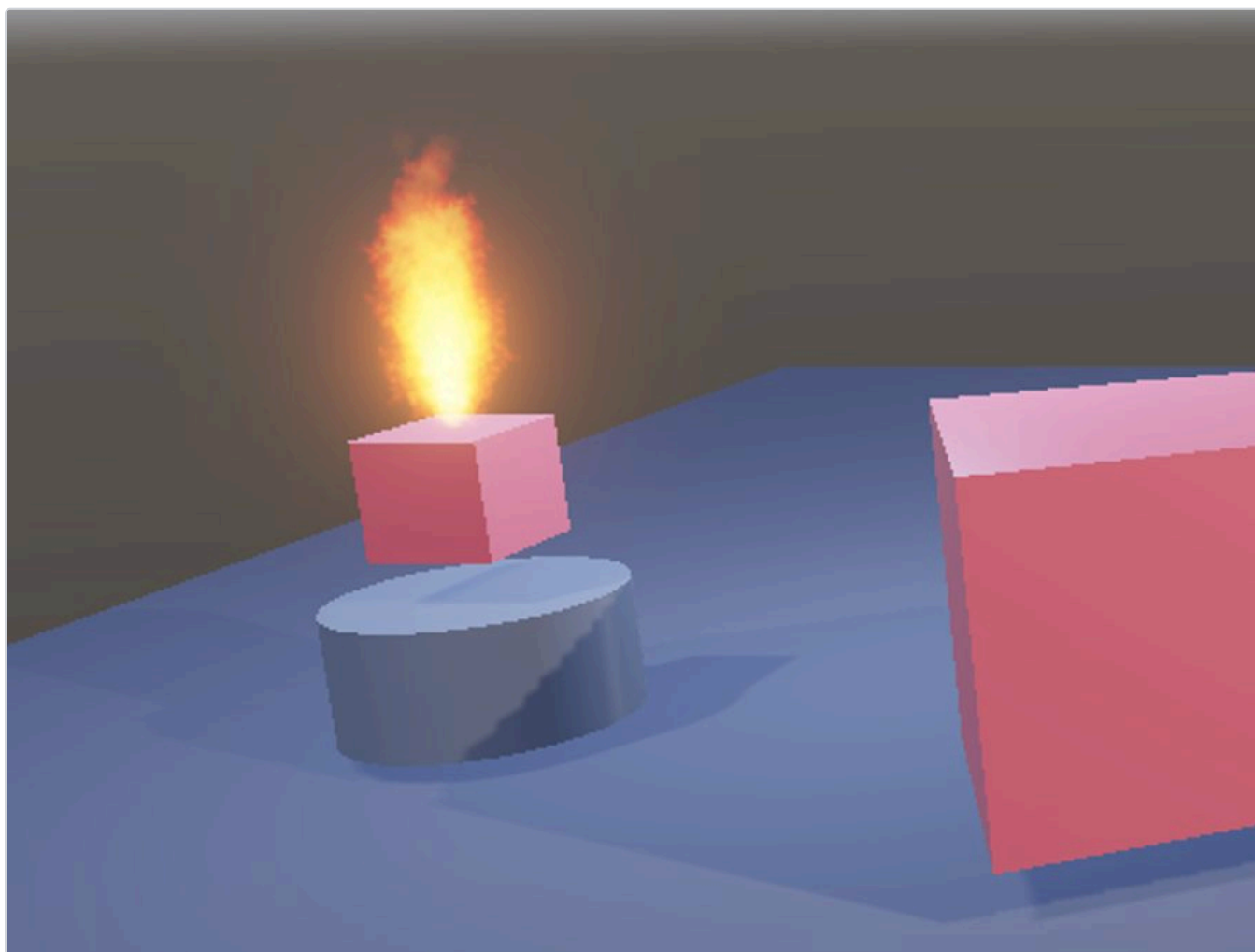


Figure 11. The current end of the road: a textured, curve-driven, additive flame burning on the sample scene's pedestal, in local space so it rides whatever carries it.

Open, in priority order

1. **Text and game UI.** The largest gap by distance, and the one the mini game hit hardest — it could not display a score, a label or a prompt, and communicated entirely through a light and four sounds.
2. **A per-frame script hook.** The second largest, and analysed in @@@@§10.1: every other presentation system already runs per frame, and scripts moving on the simulation step actively fight the interpolation.

3. **GPU alpha self-sorting.** A bitonic sort in compute keyed on view depth; because the default pool is 1000 particles, anything up to 2048 sorts inside a single workgroup in shared memory — one dispatch. The measurement in @@§9.9 is the argument for it.
4. **Particle collision and sub-emitters**, both scoped and deliberately deferred.

What a reader should take away

Not the feature list. Three habits:

- **Order the work by what other work will be described in terms of.** Identity, serialization, reflection and the two abstractions came first, and everything else was cheaper because of it.
- **Build something real with the engine before extending it.** One small game found six holes that no amount of internal testing had.
- **Measure, and construct references that cannot be wrong.** Most of the hard-won entries in §9 were found by a measurement, and most of the ones that took longest were the ones where the picture looked plausible.

12 Walkthroughs — building it yourself

Chapter 4 argues an order. This chapter walks it. Eight milestones, each one ending with something running on your machine and a check that proves it — because a milestone you cannot demonstrate is a milestone you have not reached.

Each is written the same way: **what you will have** when it is done, **the steps in order**, **the traps** that cost time here specifically, and **the proof** — the thing you run to know it worked. Follow them in sequence; each leans on the last, and that dependency is the whole argument of chapter 4.

None of this is RageV-specific. The libraries named are the ones this engine happened to pick; the shape of each milestone would be the same with different ones.

12.1 Milestone 0 — a window, a device, and a triangle

You will have

An executable that opens a window, initialises a graphics device, clears to a colour you chose, draws one triangle, and exits cleanly with code 0.

This is deliberately small, and it is not the "hello triangle" of a tutorial — the difference is the last four words. Exiting cleanly means every object you created was destroyed in an order the API accepts, and getting that right now is much cheaper than retrofitting it once there are forty kinds of object.

Steps

1. **Build system first.** CMake, with your dependencies vendored into the repository rather than expected on the machine. You will thank yourself the first time you clone onto a second computer. Aim for: clone, configure, build, run — no prerequisites beyond a compiler.
2. **A window.** GLFW is the reasonable default. Do *not* let anything but one wrapper class touch it (§5.2 explains what happens when a second component initialises it independently).
3. **A device.** Vulkan: instance → physical device → logical device → queue → swapchain. OpenGL: a context and a function loader. Enable the validation layer or debug callback *now*, and make its output impossible to ignore.
4. **One triangle**, with vertices in the shader rather than a buffer (`gl_VertexIndex` and a constant array). You are testing the device and the pipeline, not vertex input, and the fewer moving parts the better.
5. **Clean shutdown.** Destroy in reverse order of creation; wait for the device to be idle first.

The code

Start with the build. Vendor everything, so a clone builds with no prerequisites:

```
# CMakeLists.txt
cmake_minimum_required(VERSION 3.20)
project(MyEngine LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)

add_subdirectory(vendor/glfw)           # checked in, not found on the system

add_executable(MyEngine src/main.cpp)
target_link_libraries(MyEngine PRIVATE glfw)
```

Then the loop. The shape below is the one worth copying: **a frame counter and an exit condition from the start**, because everything you automate later depends on the program being able to stop by itself.

```
// src/main.cpp
int main(int argc, char** argv)
{
    // --frames=N runs N frames and exits. Not a debug hack: it is what
    // makes a screenshot check, a benchmark and a CI run possible later.
    int frameLimit = -1;
    for (int i = 1; i < argc; i++)
        if (std::string_view(argv[i]).starts_with("--frames="))
            frameLimit = std::atoi(argv[i] + 9);

    if (!glfwInit())
        return 1;

    // No GL context: the device owns presentation. Change this line and
    // the hints below when you add the second backend in milestone 3.
    glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
    GLFWwindow* window = glfwCreateWindow(1280, 720, "MyEngine", nullptr, nullptr);
    if (!window)
        return 1;

    Device device(window);           // instance, device, queue, swapchain
    Pipeline triangle(device, "assets/triangle.glsl");

    int frame = 0;
    while (!glfwWindowShouldClose(window))
    {
        glfwPollEvents();

        if (CommandList* cmd = device.BeginFrame()) // null = skip this frame
        {
            cmd->BeginRenderPass(device.Backbuffer(), /*clear*/ { 0.1f, 0.1f, 0.12f, 1 });
            cmd->BindPipeline(triangle);
            cmd->Draw(3);                      // no vertex buffer -- see below
            cmd->EndRenderPass();
            device.EndFrame();
        }

        if (frameLimit >= 0 && ++frame >= frameLimit)
            break;
    }

    device.WaitIdle();                // before anything is destroyed
    glfwDestroyWindow(window);
    glfwTerminate();
    return 0;                         // the exit code is part of "it works"
}
```

The triangle carries no vertex buffer. Three vertices from the index alone means the only things under test are the device, the pipeline and the swapchain:

```
// assets/triangle.glsl -- vertex stage
void main()
{
    // A triangle that covers the middle of the screen, from gl_VertexIndex.
    vec2 p[3] = vec2[3](vec2(0.0, -0.5), vec2(0.5, 0.5), vec2(-0.5, 0.5));
    gl_Position = vec4(p[gl_VertexIndex], 0.0, 1.0);
}
```

Note what `BeginFrame` returning null means, because it is not an error: a minimised or resizing window has no image to render into, and the correct response is to skip the frame entirely — **including not calling `EndFrame`**. Handling it now costs one `if`; discovering it later costs an afternoon of "why does resizing crash".

Traps

- **Do not run without validation.** Vulkan will happily produce a black screen for a dozen different reasons, and the layer names all of them precisely. Turning it on later means wading through a hundred accumulated messages.
- **Verify by exiting, not by killing.** If you close the window with the debugger's stop button, no destructors run and every lifetime bug you are about to write stays invisible. Make exit code 0 part of what "it works" means from day one.

Proof

The window shows your triangle, the console shows zero validation messages, and the process exits with code 0 when you close it. Add a flag now — `--frames=N` — that runs N frames and exits. Everything you automate later depends on the program being able to stop on its own.

12.2 Milestone 1 — the scene model, with nothing drawn

You will have

A scene you can build in code, save to a text file, load back, and serialize again to a *byte-identical* file. With undo. And still nothing on screen.

This is the milestone people skip, and skipping it is the decision that caps a project (§4.2). Resist the urge to draw anything.

Steps

1. **Pick an ECS** — EnTT is the usual answer in C++ — and wrap it in a `Scene`. Do not let the ECS type escape into the rest of the engine; you want to be able to change it.
2. **UUIDs.** A 64-bit random identity component on every entity, assigned at creation. Add a `Scene::GetEntityByUUID`. From here on, *every* reference that outlives a frame is a UUID (§8.1).
3. **Hierarchy.** A relationship component holding a parent UUID. Rebuild children on load; do not serialize them. Write `UpdateWorldTransforms` as one top-down pass and resist the dirty flag (§8.1).
4. **Reflection.** A registry describing each component's fields — name, type, offset — reachable generically. This is the piece that makes the next two steps small instead of enormous (§5.3).

5. **The serializer, written against the registry.** Walk components, walk fields, write. Then read. YAML is a good choice: a human can diff it and fix it.
6. **The undo stack.** Commands with execute and undo. Do it now, because every editor action you write later must go through it, and retrofitting means rewriting all of them.

The code

Identity first, and it is smaller than you expect:

```
// A 64-bit random identity. Not a counter: counters collide the moment two
// people author scenes separately, and there is no way to notice.
class UUID
{
public:
    UUID() : m_Value(RandomU64()) {}
    UUID(uint64_t value) : m_Value(value) {}
    operator uint64_t() const { return m_Value; }
    bool IsValid() const { return m_Value != 0; }
private:
    uint64_t m_Value;
};

struct IDComponent          { UUID ID; };
struct RelationshipComponent
{
    UUID Parent = UUID(0);
    std::vector<UUID> Children;    // rebuilt on load, never serialized
};
```

Then the description that everything else is generated from. This is the piece that makes the serializer and the inspector small:

```

enum class FieldType { Bool, Int, Float, Vec3, Vec4, String, Asset, Enum };

struct FieldDesc
{
    const char* Name;           // the key on disk -- never change it casually
    FieldType   Type;
    uint32_t    Offset;        // bytes into the component
};

struct ComponentDesc
{
    const char* Name;
    std::vector<FieldDesc> Fields;

    // Type-erased access, written once here rather than per component.
    std::function<void*(Entity)> TryGet;
    std::function<void*(Entity)> Add;
};

// Registration. The macro is only computing an offset; resist making it
// clever, because a reader has to trust it.
#define FIELD(T, member, type) \
    FieldDesc{ #member, type, (uint32_t)offsetof(T, member) }

ComponentDesc desc;
desc.Name = "TransformComponent";
desc.Fields = {
    FIELD(TransformComponent, Position, FieldType::Vec3),
    FIELD(TransformComponent, Rotation, FieldType::Vec3),
    FIELD(TransformComponent, Scale,    FieldType::Vec3),
};

```

Now the serializer never names a field. It walks the description — which is why adding a field to a component is one line rather than four edits in four files:

```

void WriteEntity(Emitter& out, Entity entity)
{
    out << Key("EntityID") << Value((uint64_t)entity.GetUUID());

    for (const ComponentDesc& component : Registry::All())
    {
        void* data = component.TryGet(entity);
        if (!data)
            continue;

        out << Key(component.Name) << BeginMap;
        for (const FieldDesc& field : component.Fields)
        {
            void* value = (char*)data + field.Offset;
            switch (field.Type)
            {
                case FieldType::Float: out << Key(field.Name) << *(float*)value; break;
                case FieldType::Vec3:  out << Key(field.Name) << *(Vec3*)value; break;
                // ... one case per FieldType, and only ever here
            }
        }
        out << EndMap;
    }
}

```

And the test that will earn its keep for the rest of the project:

```

void CheckRoundTrip()
{
    auto scene = BuildFixture();           // hierarchy, several component types

    const std::string first = Serializer(scene).ToString();

    auto reloaded = std::make_shared<Scene>();
    Serializer(reloaded).FromString(first);
    const std::string second = Serializer(reloaded).ToString();

    // Not "it loaded" -- byte equality. Anything the writer emits that the
    // reader drops, or reorders, shows up here and nowhere else.
    Check(first == second, "a scene survives a save/load/save unchanged");
}

```

Traps

- **Field order must be fixed and explicit**, not whatever a hash map iterates. It is the file format.
- **Decide what a missing field means** on load, once: an older file lacking a field should get the default rather than failing. That is what lets you add fields without breaking every scene.

Proof

The round-trip test: build a scene in code with a hierarchy and several component types, serialize to a string, deserialize, serialize again, and assert the two strings are equal. Twenty lines, and it will catch more real bugs over the project's life than anything else you write.

12.3 Milestone 2 — the loop that everything else is written against

You will have

A fixed-timestep simulation with an interpolation alpha, and a demonstration that rendering is smooth when the simulation rate and the display rate disagree.

Steps

1. **The accumulator loop** exactly as in §2.13, including the clamp. Expose the fixed rate as configuration, not a constant.
2. **Two update callbacks**, distinguished by name so nobody can confuse them: one per fixed step, one per frame.
3. **Keep the previous simulation state**. Whatever moves needs its position from the last step retained, so rendering can blend. This is the part that is annoying to add later, because by then everything writes its state in place.
4. **Expose the alpha** from the application, and use it when rendering.

The code

```
void Application::Run()
{
    const float fixedDelta = 1.0f / (float)m_FixedHz;    // configuration, not a constant
    float accumulator = 0.0f;
    double previousTime = GetTime();

    while (m_Running)
    {
        const double now = GetTime();
        float frameTime = (float)(now - previousTime);
        previousTime = now;

        // Clamped, or a two-second stall tries to simulate two seconds of
        // physics in one frame, takes even longer, and never recovers.
        // Forgiving the debt is the standard, correct choice.
        frameTime = std::min(frameTime, 0.25f);
        accumulator += frameTime;

        Input::Update();                                // once per frame, not per step

        while (accumulator >= fixedDelta)                // zero or more times
        {
            for (Layer* layer : m_Layers)
                layer->OnFixedUpdate(fixedDelta);

            Input::EndFixedStep();                       // consume edges here, see below
            accumulator -= fixedDelta;
        }

        // How far we are into the step that has not happened yet.
        m_InterpolationAlpha = accumulator / fixedDelta;

        if (CommandList* cmd = m_Device->BeginFrame())
        {
            for (Layer* layer : m_Layers)
                layer->OnUpdate(frameTime);              // presentation: real elapsed time
            m_Device->EndFrame();
        }
    }
}
```

Two things in that loop are easy to get wrong and hard to notice.

The two callbacks take different values. `OnFixedUpdate` gets the *fixed* delta — the same number every call — and `OnUpdate` gets the real frame time. Passing the frame time to both is a bug that produces plausible motion and non-reproducible physics.

Input edges must survive a frame with no steps. A key pressed and released between two simulation steps must still be seen:

```
// A press is latched when it happens and cleared by the step that consumed
// it -- not at the end of the frame. A frame that runs no steps carries the
// press forward, so it is never missed; a frame that runs three does not
// deliver it three times.
```

```
bool Input::WasActionPressed(const std::string& action)
{
    return s_Pressed.contains(action);
}
```

```
void Input::EndFixedStep()
{
    s_Pressed.clear();
}
```

Then the interpolation itself. Whatever the simulation moves needs its previous state kept, so rendering can blend between the last two:

```
// Called once per FRAME, not per step.
void PhysicsWorld::SyncTransforms(Scene& scene, float alpha)
{
    for (auto& [entity, body] : m_Bodies)
    {
        const Vec3 position = Lerp(body.PreviousPosition, body.CurrentPosition, alpha);
        const Quat rotation = Slerp(body.PreviousRotation, body.CurrentRotation, alpha);
        scene.GetTransform(entity).Set(position, rotation);
    }
}
```

Slerp, not lerp, for the rotation (§2.6). Linearly interpolating two quaternions and normalising is close enough at small angles and visibly wrong at large ones — it moves at a non-constant angular speed.

Traps

- **Zero steps in a frame is normal**, not an error. At 240 Hz with a 60 Hz simulation it happens three frames in four.
- **Input events must survive a frame with no steps.** A key pressed and released between two simulation steps must still be seen by one of them, or inputs are silently dropped at high frame rates. Latch edges and clear them at the end of the step that consumed them.

Proof

Move something at constant velocity. Run the simulation at 10 Hz and render at whatever your display does. Without interpolation you will see ten discrete jumps a second; with it, smooth motion. If it looks the same either way, your alpha is not being used.

12.4 Milestone 3 — the RHI, proven by a second backend

You will have

Your triangle from milestone 0, rendering identically through two graphics APIs, with the renderer code unaware of which one is running.

This is the milestone that feels like a detour and is not. §7.1 explains why designing it against one API does not work: whatever you have will look like the shape of the problem.

Steps

1. **Write the interface from the nouns:** device, buffer, texture, shader, pipeline, resource set, sampler, command list. Nothing that mentions a mesh or a material.
2. **Port your existing backend behind it.** No new features. Getting back to the same triangle is success.
3. **Now write the second backend.** This is where the design is tested. Each time you are tempted to emulate a concept the second API lacks, ask whether the abstraction is at the wrong level (§7.1).
4. **Shader reflection.** Compile GLSL to SPIR-V, reflect the bindings and vertex inputs, and build pipeline layouts from that rather than from hand-written descriptions that can disagree with the shader.
5. **Frames in flight, honestly, in both** (§7.2). Including the fence in the backend that does not need one.

The code

This is the header the rest of the engine will be written against, so it is worth seeing in full. Notice what is *absent*: no mesh, no material, no camera, no scene.

```

// RHIDevice.h -- the only file the renderer includes to talk to a GPU.
enum class Backend { Vulkan, OpenGL };

struct DeviceDesc
{
    Backend          Backend          = Backend::Vulkan;
    NativeWindowHandle Window         = nullptr;
    uint32_t         Width            = 1600;
    uint32_t         Height           = 900;
    bool             VSync            = true;
    bool             EnableValidation = false;
    // Exposed, not hidden. It is a Vulkan concept that OpenGL has no
    // equivalent for -- and hiding it means either paying Vulkan's cost
    // everywhere or writing buffers the GPU is still reading.
    uint32_t         FramesInFlight   = 2;
};

class RHIDevice
{
public:
    virtual ~RHIDevice() = default;
    static Scope<RHIDevice> Create(const DeviceDesc& desc);

    virtual Backend          GetBackend() const = 0;
    virtual const DeviceCaps& GetCaps()      const = 0;

    // Returns null when the frame must be skipped -- swapchain out of
    // date, or a minimised window. EndFrame must not be called then.
    virtual RHICommandList* BeginFrame() = 0;
    virtual void             EndFrame()   = 0;
    virtual void             WaitIdle()   = 0;

    // Resources. Nouns that exist in both APIs, even where only one
    // names them: OpenGL has no pipeline object, but it has pipeline
    // state, and an object that records it is honest rather than faked.
    virtual Ref<RHIBuffer>      CreateBuffer(const BufferDesc&) = 0;
    virtual Ref<RHITexture>     CreateTexture(const TextureDesc&) = 0;
    virtual Ref<RHIShader>      CreateShader(const CompiledShader&) = 0;
    virtual Ref<RHIPipeline>     CreatePipeline(const GraphicsPipelineDesc&) = 0;
    virtual Ref<RHISampler>      CreateSampler(const SamplerDesc&) = 0;
    virtual Ref<RHIResourceSet> CreateResourceSet(const Ref<RHIPipeline>&, uint32_t set) = 0;

    // A one-shot list, submitted and waited on, outside the frame loop.
    // It stalls by construction -- which is what makes it right for asset
    // upload and wrong on the frame path, and what lets a headless test
    // run real GPU work with no frame loop to borrow.
    virtual void ExecuteImmediate(const std::function<void(RHICommandList&)>&) = 0;
};

```

A pipeline is described rather than commanded. Everything the two APIs disagree about is a *field* here, which is what lets each implement it in its own way:

```

struct GraphicsPipelineDesc
{
    std::string      Name;           // shows up in validation messages -- set it
    Ref<RHIShader>    Shader;
    PrimitiveTopology Topology = PrimitiveTopology::TriangleList;
    RasterizerState   Rasterizer;
    DepthStencilState DepthStencil;
    BlendPreset       Blend = BlendPreset::None;

    // What this pass writes. Vulkan bakes these into the pipeline; OpenGL
    // keeps them to validate against the bound framebuffer. Either way the
    // renderer states them once.
    std::vector<Format> ColorFormats;
    Format               DepthFormat = Format::Undefined;
};

```

And the barrier, which is the clearest example of designing at the level of **intent** because neither API's vocabulary survives translation:

```

// Not an access mask (Vulkan) and not a barrier bit (OpenGL). A *use*,
// which both can implement without lying -- and which a reader can
// understand without knowing either API.
enum class BufferSync { ComputeWrite, ComputeRead, VertexRead, IndexRead, TransferWrite };

virtual void BufferBarrier(const Ref<RHIBuffer>& buffer,
                          BufferSync from, BufferSync to) = 0;

```

Finally, the shape of the two implementations. They differ more than you might expect for the same three lines of caller code:

```

// --- Vulkan: state is baked into an object at creation -----
VulkanPipeline::VulkanPipeline(const GraphicsPipelineDesc& desc)
{
    VkPipelineColorBlendAttachmentState blend = ToVulkan(desc.Blend);
    VkPipelineDepthStencilStateCreateInfo depth = ToVulkan(desc.DepthStencil);
    // ... twenty more structs, then one vkCreateGraphicsPipelines call.
    // Expensive once, free at draw time.
}

void VulkanCommandList::BindPipeline(const Ref<RHIPipeline>& pipeline)
{
    vkCmdBindPipeline(m_Buffer, VK_PIPELINE_BIND_POINT_GRAPHICS, Handle(pipeline));
}

// --- OpenGL: the same state, applied when it is bound -----
void OpenGLCommandList::BindPipeline(const Ref<RHIPipeline>& pipeline)
{
    const GraphicsPipelineDesc& desc = pipeline->GetDesc();
    glUseProgram(ProgramOf(pipeline));
    desc.DepthStencil.DepthTestEnable ? glEnable(GL_DEPTH_TEST) : glDisable(GL_DEPTH_TEST);
    glDepthMask(desc.DepthStencil.DepthWriteEnable ? GL_TRUE : GL_FALSE);
    ApplyBlend(desc.Blend);
    // Cheap each time, and nothing is validated ahead of the draw.
}

```

Both are honest implementations of "bind this pipeline". Neither emulates the other, and that is the test from §7.1 being passed.

Traps

- **Image origin.** Vulkan's first row is the top, OpenGL's is the bottom. Decide now how you hide it — a negative-height viewport is the usual answer — and write down that texture *reads* are not covered by it (§7.7).
- **Depth range.** Standardise on 0..1 and call `glClipControl` (§2.4).
- **Reflected vector widths.** A `uvec4` is 16 bytes, and a reflection that reports a one-component format will describe it as 4 (§8.4).

Proof

Screenshot the same scene on both backends and compare the files. They will not be bit-identical — different rasterisation rules — but they should be visually indistinguishable, and a difference metric should be near zero. Automate this comparison now; §10 explains why it is the check that catches the bugs nothing else does.

12.5 Milestone 4 — the render graph and a multi-pass frame

You will have

A frame built from declared passes with pooled targets: scene into an HDR buffer, then a tonemap pass to the backbuffer. Two passes, which is the point — build the graph *before* you need it.

Steps

1. **A target description** — size, format, whether it has depth — and a pool keyed by that shape.
2. **A pass:** a name, a declare callback, an execute callback (§6.3).
3. **Compile:** resolve which targets are needed, allocate from the pool, and *refuse* malformed frames. A target created and never written is an error, not a warning.
4. **Port your existing frame onto it.** Scene → HDR, tonemap → backbuffer.

The code

A target is described, never allocated by hand. The description *is* the pool key, which is what makes reuse automatic:

```
struct RGTargetDesc
{
    std::string Name;                                // for debugging only
    uint32_t Width = 0, Height = 0;                  // 0 = follow the frame
    Format Color = Format::R16G16B16A16_SFLOAT;
    Format Depth = Format::Undefined;                 // Undefined = no depth

    // Two targets can share one allocation only if they agree here. Miss a
    // field and the pool hands back a target of the wrong shape, which
    // presents as one pass mysteriously reading another's contents.
    bool SameShape(const RGTargetDesc& other) const
    {
        return Width == other.Width && Height == other.Height
            && Color == other.Color && Depth == other.Depth;
    }
};
```

A pass is a name and two lambdas. The first says what it touches; the second draws. The split is the entire mechanism — the graph can plan because it can ask about the frame before executing it:

```
using RGRResource = uint32_t;           // an index, not a pointer

RGRResource sceneHDR = graph.CreateTarget({ .Name = "SceneHDR",
                                             .Depth = Format::D32_SFLOAT });
RGRResource output   = graph.Backbuffer();

graph.AddPass("Scene",
  [&](RGPassBuilder& builder)           // declare
  {
    builder.Write(sceneHDR);
    builder.SetClearColor({ 0.1f, 0.1f, 0.12f, 1.0f });
  },
  [&](RGPassContext& context)           // execute
  {
    Renderer3D::BeginScene(camera);
    DrawEverything();
    Renderer3D::EndScene();
  });

graph.AddPass("Tonemap",
  [&](RGPassBuilder& builder)
  {
    builder.Write(output);
    builder.Sample(sceneHDR);           // declares the dependency
  },
  [&](RGPassContext& context)
  {
    // Color(resource) resolves to whatever the pool gave that target.
    PostProcess::Tonemap(context.Color(sceneHDR));
  });

if (!graph.Compile())                   // refuse a malformed frame
  RV_CORE_ERROR("frame: {0}", graph.Errors()[0]);
else
  graph.Execute(*commandList);
```

Compile is where the graph earns its place. It is short, and every check in it corresponds to a bug that would otherwise be a black screen with no message:

```

bool RenderGraph::Compile()
{
    m_Errors.clear();

    for (Pass& pass : m_Passes)
        for (RGAttachment& attachment : pass.Attachments)
            if (attachment.Index >= ColorCountOf(pass.Target))
                m_Errors.push_back(pass.Name + " binds attachment "
                                    + std::to_string(attachment.Index)
                                    + " of a target that does not have one");

    for (const Target& target : m_Targets)
        if (!IsWritten(target))
            m_Errors.push_back(target.Desc.Name + " is created and never written");

    // Then hand out real render targets, reusing any pooled one whose
    // shape matches and whose last reader has already run.
    for (Target& target : m_Targets)
        target.Actual = m_Pool.Acquire(target.Desc);

    return m_Errors.empty();
}

```

Traps

- **Build pipelines against what the *pass* writes**, not against the final output format. Getting this backwards builds every pipeline against the wrong attachment format (§6.2).
- **An even number of fullscreen passes hides a flip bug** (§7.7). With exactly two passes you are in the hiding case, so verify orientation deliberately: render something visibly asymmetric.

Proof

The picture is unchanged, and the graph reports two passes and reuses targets across frames. Then add a third pass — a colour tint, anything — and notice that it took a declaration and nothing else. That is what you built it for.

12.6 Milestone 5 — it starts to look real

You will have

Physically-based shading, an environment that lights the scene, and shadows. This is the milestone where the engine stops looking like a demo.

Order matters within it, because each step makes the next one's errors visible.

Steps

1. **Linear colour throughout** (§2.8). sRGB textures decoded on read, lighting in a floating-point buffer, tonemap at the end. Do this *first*: doing it after you have tuned materials means retuning all of them.
2. **Cook-Torrance with metallic/roughness** (§2.9). Test with a grid of spheres sweeping roughness on one axis and metallic on the other — the standard image, and it makes every mistake obvious.
3. **A sky**, even a gradient. Metals need something to reflect or they are black.
4. **Image-based lighting** (§7.6): irradiance cube, prefiltered specular chain, BRDF lookup table. Your sphere grid should now look like it is somewhere.

5. **Shadows** (§7.5): depth from the light, then the bias work, then PCF, then cascades. In that order, and expect the bias to take longer than the rest combined.

The code

Linear colour first, and it is mostly a matter of asking for the right texture format — the hardware does the conversion for free:

```
// A colour texture is a picture and is stored gamma-encoded. Ask for an  
// _SRGB format and the sampler linearises on read, at no cost.  
TextureDesc albedo;  
albedo.Format = Format::R8G8B8A8_SRGB;      // pictures  
  
// A roughness or normal map is DATA, not a picture. Decoding it would  
// corrupt the values.  
TextureDesc roughness;  
roughness.Format = Format::R8G8B8A8_UNORM; // data
```

Then the BRDF. The three terms of §2.9, written out — this is the whole of physically based specular, and it is shorter than most people expect:

```

// D -- how many microfacets are angled to reflect l into v.
// This is what "roughness" controls, and it is the shape of the highlight.
float DistributionGGX(vec3 N, vec3 H, float roughness)
{
    float a2 = roughness * roughness * roughness * roughness;
    float NdotH = max(dot(N, H), 0.0);
    float d = NdotH * NdotH * (a2 - 1.0) + 1.0;
    return a2 / (PI * d * d);
}

// F -- everything is more reflective at grazing angles. Schlick's
// approximation; F0 is the reflectance looking straight on.
vec3 FresnelSchlick(float cosTheta, vec3 F0)
{
    return F0 + (1.0 - F0) * pow(clamp(1.0 - cosTheta, 0.0, 1.0), 5.0);
}

// G -- microfacets shadow and mask each other at glancing angles.
float GeometrySmith(vec3 N, vec3 V, vec3 L, float roughness)
{
    float k = (roughness + 1.0) * (roughness + 1.0) / 8.0;
    float gv = max(dot(N, V), 0.0) / (max(dot(N, V), 0.0) * (1.0 - k) + k);
    float gl = max(dot(N, L), 0.0) / (max(dot(N, L), 0.0) * (1.0 - k) + k);
    return gv * gl;
}

vec3 CookTorrance(vec3 N, vec3 V, vec3 L, vec3 albedo,
                  float metallic, float roughness, vec3 radiance)
{
    vec3 H = normalize(V + L);

    // Dielectrics reflect ~4% colourlessly; metals reflect their own
    // colour and have no diffuse term at all. One value blends the two.
    vec3 F0 = mix(vec3(0.04), albedo, metallic);

    vec3 F = FresnelSchlick(max(dot(H, V), 0.0), F0);
    float D = DistributionGGX(N, H, roughness);
    float G = GeometrySmith(N, V, L, roughness);

    vec3 specular = (D * G * F) /
        (4.0 * max(dot(N, V), 0.0) * max(dot(N, L), 0.0) + 0.0001);

    // Energy conservation: what is reflected is not also diffused.
    vec3 kD = (vec3(1.0) - F) * (1.0 - metallic);

    return (kD * albedo / PI + specular) * radiance * max(dot(N, L), 0.0);
}

```

Image-based lighting is then three lookups standing in for the integral (§7.6). Note how **roughness becomes a mip level** — the whole trick, in one line:

```

vec3 ImageBasedLight(vec3 N, vec3 V, vec3 albedo, float metallic, float roughness)
{
    vec3 R = reflect(-V, N);
    vec3 F0 = mix(vec3(0.04), albedo, metallic);
    vec3 F = FresnelSchlickRoughness(max(dot(N, V), 0.0), F0, roughness);

    // Diffuse: the hemisphere integral, precomputed per direction.
    vec3 irradiance = texture(u_IrradianceCube, N).rgb;
    vec3 diffuse = irradiance * albedo;

    // Specular: a rough surface reflects a blurred environment, and blur
    // is what mips already are -- so roughness IS the mip level, and the
    // hardware interpolates between roughnesses for free.
    vec3 prefiltered = textureLod(u_PrefilteredCube, R,
                                roughness * u_MaxReflectionLod).rgb;

    // The remaining scale and bias depend only on roughness and N.V,
    // so they are the same for every scene: one 2D lookup.
    vec2 brdf = texture(u_BRDFLut, vec2(max(dot(N, V), 0.0), roughness)).rg;

    vec3 kD = (vec3(1.0) - F) * (1.0 - metallic);
    return kD * diffuse + prefiltered * (F * brdf.x + brdf.y);
}

```

And the shadow bias, which is where the time goes. Both mechanisms of §7.5 in the code that applies them:

```

float NdotL = clamp(dot(N, L), 0.0, 1.0);
// tan of the angle to the light. Clamped, because at grazing incidence
// this runs away and an offset larger than the geometry detaches the shadow.
float slope = sqrt(1.0 - NdotL * NdotL) / max(NdotL, 0.15);
float offset = texelWorldSize * u_NormalBias * (1.0 + min(slope, 4.0));

// Move the SAMPLE along the normal -- not the depth. The other half of
// the bias is free: the shadow pass renders back faces, so each caster's
// recorded depth already sits behind it by its own thickness.
vec4 lightSpace = u_LightViewProj * vec4(worldPos + N * offset, 1.0);
vec3 coord = lightSpace.xyz / lightSpace.w;

if (coord.z > 1.0)
    return 1.0; // past the far plane: nothing recorded, so lit

// 3x3 of hardware 2x2 comparisons = 4x4 texels of coverage for 9 fetches.
float sum = 0.0;
for (int y = -1; y <= 1; ++y)
for (int x = -1; x <= 1; ++x)
    sum += texture(u_ShadowMap, vec3(coord.xy + vec2(x, y) * texel, coord.z));

return sum / 9.0;

```

Traps

- **Do not cull in the shadow pass.**
- **Whatever poses the lit pass must pose the depth pass.**
- **Generate mips in the command buffer that wrote mip 0 (§7.6), or your prefiltered chain is built from nothing.**

Proof

The sphere grid, lit only by an environment map, with correct-looking metals and a shadow that stays attached to its caster as the light rotates. Rotate the light through a full circle and watch for acne appearing at grazing angles — that is the test your bias has to pass.

12.7 Milestone 6 — scripting, and hot reload

You will have

Behaviour you can write in a script, attach to an entity, and change without restarting.

Steps

1. **Make the engine a DLL** (§5.2) if it is not already, and read that section's three rules before you start rather than after.
2. **A script base class** with lifecycle callbacks, and a registry mapping a name to a factory — the scene stores the name.
3. **A project's scripts compile into their own module**, which the engine loads. Unload it, rebuild it, load it again.
4. **Reload while playing.** The rule that works: stop the scene, swap, resume. Attempting to keep live instances across a reload is a much larger problem than it looks.

The code

The script surface a game programmer sees. Keep it small: every method here is one you must support forever, in both languages if you have two.

```
class ScriptableEntity
{
public:
    virtual ~ScriptableEntity() = default;

    virtual void OnCreate() {}
    virtual void OnUpdate(Timestep dt) {} // per FIXED STEP -- name it so
    virtual void OnDestroy() {}

    virtual void OnCollisionEnter(const Collision&) {}

protected:
    // Shorthands, so a script reads like the thing it is doing.
    Vec3 GetPosition() const;
    void Translate(const Vec3& delta);
    template<typename T> T& GetComponent();

    Entity m_Entity;
};
```

Scenes store a *name*, not a pointer, because a pointer cannot survive a save or a reload. A registry maps the name to a factory:

```

// The scene file says: Script: "Bobber". This is what turns that back
// into an object -- and what makes renaming a class break every scene
// that used it, which is worth documenting before somebody finds out.
class ScriptRegistry
{
public:
    using Factory = std::function<ScriptableEntity*>;

    static void Register(const std::string& name, Factory factory)
    {
        Entries()[name] = { std::move(factory), s_CurrentModule };
    }

    // Unregistering by module is what makes unloading possible: the
    // registry must not outlive the DLL whose functions it points at.
    static void UnregisterModule(ModuleHandle module)
    {
        std::erase_if(Entries(), [module](const auto& e)
            { return e.second.Module == module; });
    }
};

```

Then the reload, which is four steps and one rule:

```

void EditorLayer::ReloadScripts()
{
    // The rule: never swap code under live instances. Stop, swap, resume.
    // Keeping instances alive across a reload is a much larger problem than
    // it looks -- their vtable pointers refer to a DLL you are unloading.
    const bool wasPlaying = m_Playing;
    if (wasPlaying)
        StopPlay(); // destroys every script instance

    ScriptRegistry::UnregisterModule(m_GameModule);
    UnloadLibrary(m_GameModule); // now nothing points into it

    if (!BuildGameModule()) // compiler errors go to the log
    {
        RV_ERROR("build failed; scripts not reloaded");
        return; // stay stopped, do not resume old code
    }

    m_GameModule = LoadLibrary(ModulePath());
    if (wasPlaying)
        StartPlay();
}

```

If you host a managed runtime as well, the equivalent of "unload the DLL" is a collectible load context — and the important part is that unloading is a *request*:

```
// Loaded from BYTES, not a path: a path locks the file and the next
// build fails with "the process cannot access the file".
byte[] image = File.ReadAllBytes(assemblyPath);
var context = new AssemblyLoadContext("Scripts", isCollectible: true);
var assembly = context.LoadFromStream(new MemoryStream(image));

// ... later, on reload:
var weak = new WeakReference(context);
context.Unload();
for (int i = 0; i < 10 && weak.IsAlive; i++)
{
    GC.Collect();
    GC.WaitForPendingFinalizers();
}

// Unloading is a request, granted only when nothing references anything
// inside. A context that will not die means the OLD code is still running,
// and the symptom is "my change did nothing" -- which the developer will
// blame on the build, the editor and themselves, in that order, for an hour.
if (weak.IsAlive)
    Log.Warn("script context did not unload; something still holds a reference");
```

Traps

- **The class name is what scene files store**, so renaming a class breaks every scene using it. Say so in your documentation before someone finds out.
- **Unloading a module is a request**. Verify it actually happened and report it when it does not (§8.3), or "my change did nothing" becomes a recurring mystery.

Proof

Attach a script that rotates an entity. Press play. Change the rotation speed in the script, rebuild, and watch the speed change without the editor restarting.

12.8 Milestone 7 — ship something

You will have

A folder you can send to someone, which runs without your editor, your source tree, or your machine.

Do this **before** the engine feels finished (§4.6). It is a test of the whole system that nothing else performs.

Steps

1. **A project concept** — a file naming an asset root, a start scene and a few settings — so the engine is not hard-coded to one folder.
2. **A runtime executable** that opens a project and plays it, sharing all the frame-building code with the editor.
3. **A packager** that copies the executable, the engine, the assets, the shaders and the scripts into an output folder.
4. **Run it from somewhere else**. Not from the build directory. Copy the folder to another location, or another machine, and run it.

The code

A project is a file, so the engine is not hard-coded to one folder:

```
# MyGame.rvproject
Name: MyGame
AssetRoot: assets
StartScene: scenes/level1.rage
FixedHz: 60
```

Every path in the engine then resolves through it, and there is exactly one function that knows how:

```
// The only place a relative asset path becomes an absolute one. When
// packaging breaks, it breaks here or in something that bypassed it.
std::filesystem::path Project::AssetPath(const std::string& relative)
{
    return s_Active->RootDirectory / s_Active->Config.AssetRoot / relative;
}
```

The runtime is then almost nothing, which is the sign the layering worked — it shares every line of frame-building code with the editor:

```
void RuntimeLayer::OnAttach()
{
    if (!Project::Load(m_ProjectPath))
    {
        RV_ERROR("could not open project");
        Application::Get().Close();
        return;
    }

    // Loading a project also loads its script modules -- one line, so that
    // the editor, the runtime and a packaged game all get them the same way.
    m_Scene = SceneSerializer::Load(Project::AssetPath(Project::Config().StartScene));
    m_Scene->OnPlayStart();
}

void RuntimeLayer::OnUpdate(Timestep ts)
{
    m_Scene->OnUpdateRuntime(ts);

    // The one difference from the editor: the frame lands on the
    // backbuffer rather than an imported viewport texture.
    FrameDesc desc;
    desc.Output = Renderer::GetBackbuffer();
    desc.DrawScene = [&](RGPassContext& c) { m_Scene->OnRender(c, m_Camera); };
    FrameGraphBuilder::BuildFrame(m_Graph, desc);
}
```

And the packager, which is a copy with a list — the value is in the list being complete rather than in the code being clever:

```

bool Package(const Project& project, const std::filesystem::path& out)
{
    std::filesystem::create_directories(out);

    Copy(RuntimeExecutablePath(), out);
    Copy(EngineLibraryPath(), out); // the .dll itself
    CopyDirectory(project.AssetDirectory(), out / "assets");
    CopyDirectory(EngineShaderDirectory(), out / "assets/shaders");
    CopyDirectory(project.BinaryDirectory(), out); // the game module
    CopyDirectory(project.ManagedDirectory(), out / "managed");

    // Written beside the executable, so the runtime finds its project
    // without a command line argument.
    WriteProjectFile(out / (project.Name() + ".rvproject"), project);
    return true;
}

```

Run the result **from another directory**. Every path bug you have is waiting there, and none of them will reproduce while the build folder happens to be the working directory.

Traps

This step exists to find these, so expect them rather than being discouraged:

- Paths relative to the editor's working directory.
- Shaders loaded from the source tree.
- Assets whose handles were minted in the build output and are regenerated on a clean build.
- A pass that clears the backbuffer *after* the scene was drawn — a blank window with a perfectly healthy draw-call count, which is exactly why the RHI needs a screenshot capture (§10).

Proof

The folder runs on a machine that has never had your source tree on it.

12.9 Milestone 8 — build a game, and let it tell you what is missing

You will have

A small, complete, playable game — and a backlog written by evidence rather than by imagination.

This is the most valuable milestone in the list and the easiest to postpone. §4.8 has what it found here: six real engine holes in one session, none of which were bugs in a feature. They were all in the seams between features, and only an application that crosses the seams reaches them.

Steps

1. **Pick something genuinely small** and genuinely complete. One mechanic, a win condition, a way to restart. Complete matters more than large; the seams are in the completeness.
2. **Build it as a real project**, in the project format from milestone 7 — not as an engine sample compiled into the editor.
3. **Use the parts you are least sure about**. If you have two scripting languages, use both. If you have prefabs, spawn from one at runtime.

4. **Write down every papercut as it happens**, without stopping to fix it. The list is the deliverable.
5. **Package it and play it** from the packaged folder.

The code

There is very little engine code in this milestone, which is the point — you are *using* the engine now. What there is looks like this, and if it does not read naturally, that is your first papercut:

```
// A scoring trigger. Nothing here should be about the engine.
public class GameManager : Script
{
    private int m_Score;
    private Entity m_Beacon;

    public override void OnCreate()
    {
        // Every entity exists by now, so looking one up is safe here --
        // which is not true in a constructor. Cache it: FindByName is
        // linear, and this runs once.
        m_Beacon = Entity.FindByName("Beacon");
    }

    public override void OnTriggerEnter(Collision collision)
    {
        if (!collision.Other.Name.StartsWith("Crate"))
            return;

        m_Score++;

        // Walk the beacon from red to green as the score rises.
        float t = m_Score / 6.0f;
        m_Beacon.SetComponentField("LightComponent", "Color",
                                    $"{1 - t} {t} 0.15 1");

        PlayOneShot("audio/score.wav", 1.0f, 0.94f + Random() * 0.12f);
    }
}
```

Then keep a file open beside you and write down every friction as it happens, without stopping to fix it. The list from this engine's own attempt is in §4.8; what matters is not its contents but its *shape*. Every entry was a seam between two features that each worked:

```
// papercuts.md, written while building, fixed afterwards

- File > New Project exists but is not in the menu.           // never called
- A body added during play never simulates.                   // AddBody has no callers
- C# collision callbacks never fire from a real simulation.    // bound, unreachable
- A fresh editor has no C# until a manual build.               // nothing boots the host
- No way to draw text. Score is communicated by a light.      // biggest gap
- Camera is rigid; no per-frame hook to smooth it.            // second biggest
```

Notice that the two largest items are *absences*, not defects. A test suite cannot find an absence — only trying to build something with the engine can, and that is why this milestone is worth more than the four features you would otherwise have added instead.

Proof

Somebody who is not you plays it, from the packaged folder, without instructions. Then read your papercut list: that is your roadmap, and it will not resemble the one you would have written beforehand.

12.10 What to build after that

At this point the engine works and the question changes from "what is missing" to "what is worth adding". Three principles, all of them earned in this document:

- **Let the game rank the list.** The papercuts from milestone 8 are ordered by something real. A feature list ordered by what sounds impressive will put particles before text, and then your game still cannot say "press F".
- **Measure before and after, in the same session** (§10). Two of this engine's optimisations cut draw calls dramatically and moved the clock not at all.
- **Write the invariant down when you find it**, in terms of behaviour rather than location (§7.7). Every entry in chapter 9 is a day someone will not lose again — and the one that was written in terms of *where code lives* was violated a second time by code that lived somewhere else.

A Appendix — repository layout, build, tools

Layout

Path	Contents
RageV/src/RageV/	Engine: Core, Scene, Renderer, Asset, Physics, Audio, Animation, Particles, Managed, Project, Scripts, Math
RageV/src/Platform/	Vulkan and OpenGL RHI implementations, Windows platform
RageVEditor/	Editor application, panels, and the engine's shipped shaders
RageVRuntime/	Standalone runtime
RageVScriptCore/	The managed class library C# scripts compile against
SampleProject/ , Knockdown/	A sample project and a complete game
tools/	scenetest , rhismoke , rvdock , rvpack , and Python asset generators
docs/	Handoff, engine notes, roadmap, and the generated developer manual

Build and verify

```
cmake --build build --config Debug

build/bin/Debug/scenetest/scenetest.exe --rhi=vulkan      # 888 checks, exit 0
build/bin/Debug/scenetest/scenetest.exe --rhi=opengl
build/bin/Debug/rhismoke/rhismoke.exe 120 --rhi=vulkan

build/bin/Debug/RageVRuntime/RageVRuntime.exe \
    --rhi=vulkan --validation=on --screenshot=f.png
```

Flags worth knowing

Flag	Purpose
--rhi=vulkan opengl	Backend; also settable in ragev.ini
--validation=on	Validation layers. Off by default, ~1.5 ms/frame
--screenshot= , --screenshot-frame=	Capture a frame and exit cleanly — the basis of visual verification
--benchmark=N	Frame-time statistics with the vsync state printed beside them
--scene= , --project=	Open a specific scene or project without editing configuration
--select=	Editor: open with an entity selected, so an inspector widget can be verified
--fixed-hz=	Run the simulation at a different rate to see what breaks

A last practical note. Running an application without `--screenshot` or `--benchmark` leaves a window open indefinitely. That is not a hang, and both flags exist so that every verification run terminates on its own with an exit

code worth trusting.